Universidad de Lima

Facultad de Ingeniería — Carrera de Ingeniería de Sistemas

Trabajo de Investigación — Curso: (Área de Ingeniería de Software)

Huffman frente a LZW:

comparación teórica y experimental de algoritmos de compresión sin pérdida

---

Integrantes

Integrante A (Apellido, Nombre)
Integrante B (Apellido, Nombre)
Integrante C (Apellido, Nombre)
Integrante D (Apellido, Nombre)
Integrante E (Apellido, Nombre)

Docente

Docente del curso (Apellido, Nombre)

Lima, Perú — 2026-2

Resumen

La compresión sin pérdida reduce el costo de almacenar y transmitir datos sin alterar un solo bit de la información original. Este trabajo compara dos algoritmos clásicos que explotan redundancias de naturaleza distinta: la codificación de Huffman, un código prefijo estático que aprovecha la distribución de frecuencias de los símbolos, y el algoritmo Lempel–Ziv–Welch (LZW), un método adaptativo de diccionario que reemplaza frases repetidas por códigos. Ambos se implementaron en Java (proyecto NetBeans con Ant), con codificación canónica de Huffman y LZW de ancho variable de 9 a 16 bits con reinicio del diccionario, validados con 43 pruebas JUnit. Se derivó la complejidad temporal y espacial de cada algoritmo y se contrastó con tres experimentos: escalamiento del tiempo con el tamaño de la entrada (ajuste log–log), barrido de entropía con alfabetos uniformes y evaluación sobre los corpus Canterbury y Silesia. Los tiempos se reportan como medianas con intervalos de confianza del 95 % tras el calentamiento del compilador JIT. Las pendientes log–log, entre 0.85 y 1.08 en la corrida principal y entre 0.93 y 1.02 en una réplica independiente, confirman el crecimiento lineal de las cuatro operaciones, con asimetrías opuestas: Huffman decodifica más despacio de lo que codifica y LZW al revés. Huffman quedó siempre en $[H_0, H_0+1)$, mientras que LZW comprimió mejor en 24 de los 26 archivos del corpus (3.30 frente a 5.12 bits por byte en el agregado), pero perdió en todas las fuentes sin memoria y expandió los bytes uniformes hasta 11.00 bits por byte. El tipo de redundancia del dato decide cuál de los dos comprime mejor.

Palabras clave: compresión sin pérdida, codificación de Huffman, LZW, análisis de complejidad, algoritmia experimental, entropía.

Abstract

Lossless compression reduces the cost of storing and transmitting data without altering a single bit of the original information. This work compares two classical algorithms that exploit different kinds of redundancy: Huffman coding, a static prefix code that exploits the symbol frequency distribution, and the Lempel–Ziv–Welch (LZW) algorithm, an adaptive dictionary method that replaces repeated phrases with codes. Both were implemented in Java (a NetBeans project built with Ant), using canonical Huffman codes and variable-width LZW codes from 9 to 16 bits with dictionary reset, and validated with 43 JUnit tests. The time and space complexity of each algorithm was derived and contrasted through three experiments: time scaling with input size (log–log fit), an entropy sweep over uniform alphabets, and an evaluation on the Canterbury and Silesia corpora. Times are reported as medians with 95 % confidence intervals after JIT warm-up. Log–log slopes between 0.85 and 1.08 in the main run and between 0.93 and 1.02 in an independent replica confirm linear growth for all four operations, with opposite asymmetries: Huffman decodes more slowly than it encodes, and LZW does the reverse. Huffman always stayed within $[H_0, H_0 + 1)$, whereas LZW compressed better on 24 of the 26 corpus files (3.30 versus 5.12 bits per byte overall), but lost on every memoryless source and expanded uniform bytes to 11.00 bits per byte. The kind of redundancy in the data decides which of the two compresses better.

Keywords: lossless compression, Huffman coding, LZW, complexity analysis, experimental algorithmics, entropy.

Índice

Resumen **i**

Abstract **i**

1. Introducción **1**
1.1. Contexto . . . 1
1.2. Problema . . . 1
1.3. Objetivos . . . 1
1.4. Preguntas de investigación . . . 2
1.5. Contribuciones . . . 2
1.6. Organización del documento . . . 2

2. Estado del arte **2**
2.1. El problema: teoría de la información y codificación sin pérdida . . . 2
2.2. Algoritmos que resuelven el problema . . . 3
2.3. Comparaciones previas y metodologías de evaluación . . . 4
2.4. Síntesis y brecha . . . 5

3. Marco teórico **5**
3.1. Definiciones preliminares . . . 5
3.2. Codificación de Huffman . . . 6
3.3. Algoritmo Lempel–Ziv–Welch (LZW) . . . 8
3.4. Comparación teórica . . . 10
3.5. Hipótesis . . . 11

4. Metodología **11**
4.1. Proceso de comparación . . . 11
4.2. Implementación . . . 12
4.3. Validación de la corrección . . . 13
4.4. Conjuntos de datos . . . 14
4.5. Métricas . . . 15
4.6. Protocolo de medición . . . 15
4.7. Diseño experimental . . . 16
4.8. Entorno de ejecución . . . 16
4.9. Amenazas a la validez . . . 16

5. Resultados **17**
5.1. E1: escalamiento del tiempo con el tamaño de la entrada . . . 17
5.2. E2: barrido de entropía . . . 20
5.3. E3: corpus Canterbury y Silesia . . . 21

6. Discusión **23**
6.1. H1: escalamiento lineal . . . 23
6.2. H2: Huffman frente a la entropía . . . 23
6.3. H3: tipo de redundancia y ganador . . . 24
6.4. H4: sensibilidad del tiempo a la entropía . . . 24

7. Conclusiones **25**
7.1. Comparación entre el análisis teórico y los resultados experimentales . . . 25
7.2. Respuestas a las preguntas de investigación . . . 26

Referencias **27**

A. Reproducibilidad **29**

B. Detalle de la implementación **29**
B.1. Codificación de Huffman 29
B.2. Algoritmo LZW 30
B.3. Metodología experimental y arnés de medición 30
B.4. Generación de figuras y tablas 31

Índice de figuras

1. Árbol de Huffman para `abracadabra`; los nodos internos muestran el peso acumulado. . . . 7
2. Metodología de la comparación entre Huffman y LZW. . . . 12
3. E1: mediana del tiempo frente al tamaño de la entrada (escala log–log). . . . 17
4. E1: costo por byte (ns/byte) frente al tamaño de la entrada. . . . 19
5. E1: bits por byte frente al tamaño de la entrada. . . . 20
6. E2: bits por byte frente al tamaño del alfabeto ($n = 2^{23}$). . . . 20
7. E2: tiempo frente al tamaño del alfabeto ($n = 2^{23}$). . . . 21

Índice de tablas

1. Estudios comparativos relacionados. . . . 5
2. Longitudes y códigos para `abracadabra`. . . . 7
3. Codificación LZW de `TOBEORNOTTOBEORTOBEORNOT` (solo los pasos que emiten). . . . 10
4. Comparación teórica de las implementaciones. . . . 11
5. Paquetes del proyecto (`compression`). . . . 12
6. Batería de pruebas JUnit (43 pruebas). . . . 14
7. Conjuntos de datos. . . . 15
8. Experimentos. . . . 16
9. Entorno de ejecución (generado por `BenchmarkRunner`). . . . 16
10. E1: pendiente $\hat{k}$ del ajuste $\ln t = \hat{k} \ln n + c$, coeficiente R^2 y costo por byte en el mayor n, en la corrida principal y en la réplica. . . . 18
11. E3: bits por byte y rendimiento de codificación en los corpus Canterbury (arriba) y Silesia (abajo). . . . 22
12. Predicción teórica frente a evidencia experimental. . . . 25
13. Algoritmo recomendado según la situación, con la evidencia medida. . . . 26

Índice de algoritmos

1. Longitudes de código de Huffman (`codeLengths`). . . . 6
2. Decodificación canónica de un símbolo (`decodeSymbol`). . . . 8
3. Codificación LZW con reinicio (`compress`). . . . 9
4. Decodificación LZW (`decode`). . . . 9

Índice de listados

1. Fusión de nodos y cálculo de profundidades en `HuffmanCodec.codeLengths`. . . . 12
2. Bucle de codificación de `LzwCodec.compress`. . . . 13
3. Comandos para reproducir el estudio. . . . 29

1. Introducción

1.1. Contexto

El volumen de datos que se almacena y transmite crece de forma sostenida, y con él el costo de discos, memoria y ancho de banda. La compresión de datos reduce ese costo representando la misma información con menos bits. Cuando la reconstrucción debe ser exacta —texto, código fuente, ejecutables, registros científicos o bases de datos—, solo son aceptables los métodos *sin pérdida*, en los que la descompresión devuelve bit a bit la entrada original [1, 2]. La teoría de la información establece el límite de lo alcanzable: ningún código puede representar una fuente con menos bits por símbolo que su entropía [3].

Dos algoritmos ocupan un lugar central en la historia y en la práctica de la compresión sin pérdida. La codificación de Huffman [4] construye un código prefijo óptimo a partir de las frecuencias de los símbolos, y el algoritmo Lempel–Ziv–Welch (LZW) [5], derivado de LZ78 [6], sustituye frases repetidas por referencias a un diccionario que se construye durante la lectura. Ambos siguen vigentes: la codificación de Huffman forma parte de DEFLATE [7] y de numerosos formatos, y LZW se emplea en formatos de imagen y de archivo, y es una referencia habitual en dispositivos con recursos limitados [8, 9]. Comparar ambos enfoques permite entender dos formas complementarias de explotar la redundancia: la estadística de los símbolos aislados y la repetición de secuencias [10, 11].

1.2. Problema

Los estudios comparativos de Huffman y LZW disponibles suelen limitarse a un único dominio de datos o a archivos de pocos tamaños, reportan tiempos de una sola ejecución y no contrastan las mediciones con la complejidad teórica de los algoritmos [8, 12, 13]. En consecuencia, no queda claro en qué condiciones cada algoritmo comprime mejor, si sus tiempos crecen realmente de forma lineal con el tamaño de la entrada, ni cómo influye la entropía de la fuente. El problema que aborda este trabajo es, por tanto: *¿cómo se comparan Huffman y LZW, en tiempo de ejecución y en tasa de compresión, sobre una gran cantidad de datos de naturaleza diversa, y en qué medida el comportamiento observado coincide con el análisis teórico?*

1.3. Objetivos

Objetivo general. Comparar teórica y experimentalmente la codificación de Huffman y el algoritmo LZW en tiempo de compresión, tiempo de descompresión y tasa de compresión, contrastando las mediciones con el análisis de complejidad de cada algoritmo.

Objetivos específicos.

OE1. Implementar en Java ambos algoritmos (Huffman canónico y LZW de ancho variable de 9 a 16 bits) y verificar su corrección con una batería de pruebas unitarias que incluya ida y vuelta (*round trip*) sin pérdida en el 100 % de los casos.

OE2. Derivar la complejidad temporal y espacial de codificación y decodificación de cada algoritmo y formular hipótesis contrastables a partir de ella.

OE3. Medir el tiempo de ejecución en función del tamaño de la entrada desde 16 KiB (2^{14} bytes) hasta 16 MiB (2^{24} bytes) en cuatro escenarios de datos (hasta 8 MiB en el texto real) y estimar el exponente de crecimiento mediante regresión log–log con su coeficiente R^2.

OE4. Medir la tasa de compresión (bits por byte) frente a la entropía de orden cero en un barrido controlado de alfabetos y en los corpus estándar Canterbury y Silesia.

1.4. Preguntas de investigación

RQ1. ¿El tiempo de codificación y decodificación de Huffman y de LZW crece con el tamaño de la entrada según lo que predice su análisis de complejidad?

RQ2. ¿Cómo se relaciona la tasa de compresión de cada algoritmo con la entropía de orden cero y con el tipo de dato (fuentes sin memoria frente a datos con frases repetidas)?

1.5. Contribuciones

- Dos implementaciones documentadas y probadas con una interfaz común (`Codec`): Huffman canónico estático y LZW de ancho variable con reinicio del diccionario.
- Un análisis de complejidad de ambos algoritmos ligado a las decisiones concretas de implementación (montículo binario, decodificación canónica, tabla hash de direccionamiento abierto).
- Un protocolo experimental reproducible, con calentamiento del JIT, repeticiones, medianas e intervalos de confianza [14], y ajuste de pendientes log–log para validar la complejidad [15].
- Una evaluación sobre datos sintéticos controlados y sobre los corpus de referencia Canterbury [16] y Silesia [17].

1.6. Organización del documento

La sección 2 revisa críticamente el estado del arte: los fundamentos teóricos del problema, las familias de algoritmos que lo resuelven y los estudios comparativos previos. La sección 3 presenta el marco teórico: definiciones, ambos algoritmos con su pseudocódigo, ejemplos resueltos, el análisis de complejidad y las hipótesis H1–H4. La sección 4 describe gráficamente la metodología, la implementación, los conjuntos de datos y el protocolo de medición. La sección 5 reporta los resultados de los experimentos E1–E3, y la sección 6 los contrasta con la teoría. La sección 7 responde las preguntas de investigación. Los anexos detallan cómo reproducir el estudio y el detalle de la implementación de cada componente.

2. Estado del arte

2.1. El problema: teoría de la información y codificación sin pérdida

El punto de partida de toda la disciplina es el trabajo de Shannon [3], que modela una fuente como un proceso aleatorio y define su entropía como el número mínimo promedio de bits por símbolo con que puede representarse sin pérdida. El teorema de codificación de fuente convierte la compresión en un problema bien planteado: existe un límite inferior y la calidad de un compresor se mide por su distancia a él. Sin embargo, la entropía depende del *modelo* de la fuente: un modelo de orden cero, que trata cada byte como independiente, ignora la redundancia contenida en secuencias repetidas, mientras que modelos con contexto capturan dependencias entre símbolos consecutivos [10].

Bell, Witten y Cleary [10] propusieron una distinción que sigue organizando el campo: todo compresor combina un *modelo* que asigna probabilidades o identifica repeticiones y un *codificador* que traduce esa información en bits. Bajo esa óptica, los métodos se agrupan en *estadísticos*, que codifican símbolos según su probabilidad, y *de diccionario*, que sustituyen secuencias por referencias. Los textos de referencia de Sayood [1] y de Salomon y Motta [2] adoptan esta misma taxonomía, y la revisión de Jayasankar et al. [11] la amplía al clasificar las técnicas por calidad de reconstrucción, esquema de codificación, tipo de dato y aplicación. Una conclusión común a estas revisiones es que no existe un compresor universalmente mejor: el desempeño depende de

la coincidencia entre la redundancia que explota el método y la que contienen los datos. Esta observación motiva comparar Huffman y LZW sobre fuentes de naturaleza deliberadamente distinta.

2.2. Algoritmos que resuelven el problema

Familia de Huffman. Huffman [4] demostró que un algoritmo voraz que fusiona repetidamente los dos símbolos menos probables produce un código prefijo de longitud media mínima para una distribución conocida. Gallager [18] caracterizó estos códigos mediante la propiedad de hermanos (*sibling property*), acotó su redundancia en función de la probabilidad del símbolo más frecuente y planteó una versión adaptativa. Esa línea se completó con los algoritmos dinámicos de Knuth [19] y de Vitter [20], que actualizan el árbol tras cada símbolo y evitan la segunda pasada y la transmisión de la tabla, a cambio de un mayor costo por símbolo. Desde el punto de vista de la implementación, Moffat y Turpin [21] mostraron que basta transmitir las longitudes de los códigos si se usan *códigos canónicos*, que además permiten decodificar con tablas pequeñas; la revisión de Moffat [22] sintetiza siete décadas de estas mejoras. La limitación estructural de toda la familia es que asigna un número entero de bits a cada símbolo, por lo que pierde hasta un bit por símbolo frente a la entropía cuando la distribución está muy sesgada.

Codificación aritmética: alternativa fuera de alcance. La codificación aritmética [23] supera esa limitación al representar el mensaje completo como un intervalo y acercarse a la entropía del modelo con una fracción de bit de sobrecosto por mensaje. Constituye el estándar en compresores estadísticos avanzados, pero su comparación excede el alcance de este trabajo, centrado en los dos algoritmos clásicos de cada familia.

Familia Lempel–Ziv. Ziv y Lempel propusieron dos esquemas universales que no requieren conocer la distribución de la fuente: LZ77 [24], que codifica cada frase como una referencia a una coincidencia previa dentro de una ventana deslizante, y LZ78 [6], que construye incrementalmente un diccionario explícito de frases y cuya tasa converge asintóticamente a la entropía para fuentes estacionarias y ergódicas. Storer y Szymanski [25] formalizaron la compresión por sustitución textual y propusieron evitar referencias que no ahorran espacio, idea conocida como LZSS. Welch [5] simplificó LZ78 inicializando el diccionario con todos los símbolos del alfabeto, de modo que la salida consiste solo en índices; el resultado, LZW, es rápido y sencillo de implementar en hardware y software, pero su diccionario crece sin límite si no se gestiona.

DEFLATE: la combinación de ambas ideas. El formato DEFLATE [7] combina una etapa LZ77 con una codificación de Huffman canónica de los literales, longitudes y distancias resultantes. Su éxito práctico ilustra que ambas familias son complementarias: la etapa de diccionario elimina la repetición de frases y la etapa estadística, el sesgo en la distribución de los símbolos resultantes. DEFLATE queda fuera del alcance de este trabajo, que compara los dos mecanismos por separado.

Gestión del diccionario en LZW. Cuando el diccionario de LZW se llena, el compresor debe decidir qué hacer. Cheng et al. [26] comparan las políticas habituales: el reinicio completo (*reset*) es más rápido pero empeora la tasa de compresión, mientras que el reemplazo LRU obtiene mejor tasa a costa de más tiempo y memoria; proponen IDA-LZW, que gestiona un área de entradas poco significativas. La implementación de este trabajo adopta el reinicio, la política más simple y predecible.

2.3. Comparaciones previas y metodologías de evaluación

La tabla 1 resume los estudios que comparan directamente Huffman y LZW o variantes de ambos. Jambek y Khairi [8] los evalúan para nodos de redes de sensores inalámbricos y reportan que Huffman redujo los datos en alrededor de 43 % en promedio y fue aproximadamente cuatro veces más rápido que LZW. Fauzan et al. [12], en cambio, trabajan con archivos de texto (`.txt`, `.csv`, entre otros) y encuentran que LZW obtuvo mayor ahorro de espacio en `.txt` y `.csv` (63.85 % y 77.56 % en promedio), con tiempos de compresión que crecen con el tamaño del archivo. Btoush et al. [13] comparan LZW, Huffman y codificación de longitud fija sobre textos de longitud variable, midiendo tasa, tiempo y entropía, y observan que LZW es el peor en archivos pequeños. Bjärås [9] evalúa cinco algoritmos ligeros para series temporales en dispositivos IoT, incluidas variantes basadas en Huffman y en LZW, y concluye que las basadas en Huffman ofrecen el mejor equilibrio entre tasa, memoria y uso de CPU. Cheng et al. [26] no comparan contra Huffman, pero muestran que la política de diccionario lleno cambia por sí sola la tasa y el tiempo de LZW.

Estos resultados, aparentemente contradictorios, son coherentes con la observación de la sección 2.1: las lecturas de sensores y las series temporales tienen poca repetición de frases y una distribución de valores sesgada, lo que favorece a Huffman, mientras que el texto contiene palabras y frases recurrentes, que LZW aprovecha una vez que su diccionario se ha poblado; en archivos pequeños el diccionario no alcanza a aprender, lo que explica la desventaja reportada por Btoush et al. Ninguno de estos trabajos, sin embargo, aísla experimentalmente cada tipo de redundancia ni verifica cómo escalan los tiempos.

Respecto a la metodología, la literatura de algoritmia experimental ofrece pautas que los estudios anteriores no siguen. Johnson [27] recomienda instancias de prueba diversas y de tamaño creciente, informes reproducibles y comparaciones con la teoría. McGeoch [15] describe cómo estimar la tasa de crecimiento de un algoritmo a partir de mediciones, por ejemplo mediante la pendiente en escala log–log. Georges et al. [14] demuestran que, en Java, la compilación JIT, la recolección de basura y la variabilidad entre ejecuciones pueden invertir las conclusiones si se reporta una única medición, y proponen repeticiones con intervalos de confianza. Por último, el uso de corpus públicos como Canterbury [16] y Silesia [17] hace comparables los resultados entre estudios.

Tabla 1: Estudios comparativos relacionados.

Trabajo	Algoritmos	Datos	Métricas	Hallazgo principal
[8]	Huffman, LZW	Datos de nodos de sensores inalámbricos	Reducción de datos, tiempo	Huffman redujo los datos alrededor de 43 % en promedio y fue unas cuatro veces más rápido que LZW.
[12]	Huffman, LZW	Archivos de texto (`.txt`, `.csv`, otros)	Ahorro de espacio, tiempo	LZW logró mayor ahorro en `.txt` y `.csv` (63.85 % y 77.56 % en promedio); el tiempo creció con el tamaño del archivo.
[13]	LZW, Huffman, longitud fija	Textos de longitud variable	Tasa, tiempo, entropía	LZW fue el peor en archivos pequeños.
[9]	Cinco algoritmos ligeros (variantes Huffman y LZW, entre otros)	Series temporales IoT	Tasa, memoria, CPU	Las variantes basadas en Huffman resultaron las más equilibradas.
[26]	Políticas de diccionario de LZW; IDA-LZW	Conjuntos de prueba propios del estudio	Tasa, tiempo, memoria	El reinicio es más rápido pero comprime peor; LRU comprime mejor con más tiempo y memoria.
Este trabajo	Huffman canónico, LZW 9–16 bits	Sintéticos controlados, texto real, Canterbury y Silesia	Tiempo (mediana, IC 95 %), pendiente log–log, bpb	Ver secciones 5 y 6.

2.4. Síntesis y brecha

La revisión permite identificar tres carencias en las comparaciones previas entre Huffman y LZW. Primero, los datos son pequeños o de un único dominio (sensores, series temporales o texto), lo que impide separar el efecto del tipo de redundancia del efecto del algoritmo. Segundo, ningún estudio valida la complejidad teórica: los tiempos se reportan para unos pocos tamaños, sin estimar la tasa de crecimiento. Tercero, falta rigor estadístico: no se describen calentamiento, repeticiones ni intervalos de confianza, pese a que en entornos con JIT esto puede alterar las conclusiones [14].

Este trabajo aborda esas carencias con (i) fuentes sintéticas que aíslan cada tipo de redundancia, junto con texto real y los corpus estándar [16, 17]; (ii) tamaños de entrada que crecen geométricamente, de 2^{14} a 2^{24} bytes, y un ajuste log–log contrastado con la complejidad derivada [15]; y (iii) un protocolo con calentamiento, repeticiones, medianas e intervalos de confianza de Student, siguiendo las recomendaciones de la algoritmia experimental [27, 14].

3. Marco teórico

3.1. Definiciones preliminares

Sea $x = x_1 x_2 \cdots x_n$ una secuencia de n bytes sobre el alfabeto $\Sigma = \{0, \ldots, 255\}$, de la cual aparecen $\sigma \leq 256$ símbolos distintos. La frecuencia del símbolo s es f_s y su probabilidad empírica es $p_s = f_s/n$.

Fuente y entropía de orden cero. Un modelo de orden cero trata los símbolos como independientes e idénticamente distribuidos (i.i.d.). Su entropía empírica

$$H_0(x) = -\sum_{s\,:\,f_s>0} p_s \log_2 p_s \qquad \text{[bits por símbolo]} \tag{1}$$

es el número mínimo promedio de bits por símbolo de cualquier código que asigne a cada símbolo una palabra fija [3]. Cumple $0 \leq H_0 \leq \log_2 \sigma \leq 8$, con igualdad superior para la distribución

uniforme.

Códigos prefijo y desigualdad de Kraft–McMillan. Un código asigna a cada símbolo s una palabra binaria de longitud ℓ_s; es *prefijo* si ninguna palabra es prefijo de otra, lo que permite decodificar sin separadores. Todo código unívocamente decodificable satisface

$$\sum_{s} 2^{-\ell_s} \leq 1, \tag{2}$$

y, recíprocamente, para cualquier conjunto de longitudes que cumpla (2) existe un código prefijo con esas longitudes [1, 2]. La longitud media del código es $\bar{L} = \sum_s p_s \ell_s$.

Métricas de compresión. Si $|C(x)|$ es el tamaño en bytes de la salida comprimida, incluida la cabecera, se definen

$$\text{bpb} = \frac{8\,|C(x)|}{n}, \qquad \text{razón} = \frac{n}{|C(x)|}, \qquad \text{ahorro} = 1 - \frac{|C(x)|}{n}. \tag{3}$$

Los bits por byte (bpb) permiten comparar directamente con H_0: un valor bpb > 8 indica expansión.

3.2. Codificación de Huffman

Idea. Huffman [4] construye un árbol binario de abajo hacia arriba: parte de una hoja por símbolo, ponderada por su frecuencia, y fusiona repetidamente los dos árboles de menor peso. La profundidad de cada hoja es la longitud de su código. Se trata de un algoritmo voraz cuya elección local —fusionar los dos símbolos menos frecuentes— es óptima [28]. El algoritmo 1 muestra el cálculo de longitudes tal como lo realiza `HuffmanCodec.codeLengths`, con un montículo binario de mínimos y desempate por orden de creación del nodo para que el resultado sea determinista.

Algoritmo 1 Longitudes de código de Huffman (`codeLengths`).

Entrada: frecuencias $f_0, \ldots, f_{255}$
Salida: longitudes $\ell_0, \ldots, \ell_{255}$ ($\ell_s = 0$ si $f_s = 0$)

```
Q ← montículo de mínimos con una hoja por cada s con f_s > 0, clave w(s) = f_s
si |Q| = 1 entonces
    devolver ℓ_s = 1 para el único símbolo presente
fin si
mientras |Q| > 1 hacer
    a ← ExtraerMín(Q);  b ← ExtraerMín(Q)
    crear nodo z con w(z) = w(a) + w(b);  padre(a) ← z;  padre(b) ← z
    Insertar(Q, z)
fin mientras
para cada nodo v en orden decreciente de creación hacer
    prof(v) ← prof(padre(v)) + 1                ▷ la raíz tiene profundidad 0
fin para
devolver ℓ_s = prof(s) para cada hoja s
```

Optimalidad y cota de entropía. Entre todos los códigos prefijo para una distribución dada, el de Huffman minimiza $\bar{L}$ [4, 28]. Como su árbol es completo (todo nodo interno tiene dos hijos), para $\sigma \geq 2$ la desigualdad (2) se cumple con igualdad. Además,

$$H_0 \;\leq\; \bar{L} \;<\; H_0 + 1, \tag{4}$$

la cota inferior por el teorema de Shannon [3] y la superior porque el código con longitudes $\lceil -\log_2 p_s \rceil$ satisface (2) y Huffman no puede ser peor [1]. Gallager [18] refinó la cota superior en función de la probabilidad del símbolo más frecuente.

Códigos canónicos. Las longitudes bastan para reconstruir el código si las palabras se asignan de forma *canónica*: en orden creciente de longitud y, dentro de cada longitud, en orden creciente de símbolo, como especifica DEFLATE [7]. Con c_ℓ el número de códigos de longitud ℓ, el primer código de longitud ℓ es

$$\mathit{first}_1 = 0, \qquad \mathit{first}_\ell = \big(\mathit{first}_{\ell-1} + c_{\ell-1}\big) \cdot 2, \tag{5}$$

y los símbolos de esa longitud reciben códigos consecutivos a partir de él. Por eso la implementación guarda en la cabecera solo 256 longitudes de un byte, en lugar del árbol.

Ejemplo: `abracadabra`. La cadena tiene $n = 11$ y frecuencias a:5, b:2, r:2, c:1, d:1. El algoritmo fusiona primero c y d (peso 2), luego b y r (peso 4), después ambos subárboles (peso 6) y finalmente el resultado con a (peso 11). La figura 1 muestra el árbol y la tabla 2 las longitudes y los códigos. El código canónico depende solo de las longitudes, por lo que puede diferir del que se lee en el árbol sin cambiar $\bar{L}$.

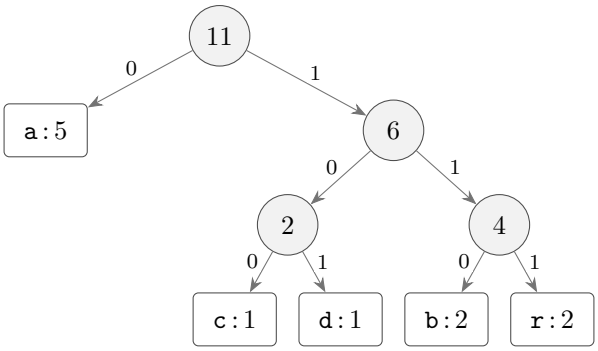


Figura 1: Árbol de Huffman para `abracadabra`; los nodos internos muestran el peso acumulado.

Tabla 2: Longitudes y códigos para `abracadabra`.

Símbolo	Frecuencia	Longitud ℓ_s	Código del árbol	Código canónico
`a`	5	1	`0`	`0`
`b`	2	3	`110`	`100`
`c`	1	3	`100`	`101`
`d`	1	3	`101`	`110`
`r`	2	3	`111`	`111`

Con $c_1 = 1$, $c_2 = 0$ y $c_3 = 4$, la ecuación (5) da $\mathit{first}_1 = 0$, $\mathit{first}_2 = (0 + 1) \cdot 2 = 2$ y $\mathit{first}_3 = (2 + 0) \cdot 2 = 4 = \texttt{100}_2$, de modo que b, c, d, r reciben `100`, `101`, `110` y `111`. La cadena se codifica como `0 100 111 0 101 0 110 0 100 111 0`, es decir, 23 bits frente a 88 bits sin comprimir. Así, $\bar{L} = 23/11 \approx 2.091$ y $H_0 \approx 2.040$, lo que verifica (4); además $2^{-1} + 4 \cdot 2^{-3} = 1$, la igualdad de Kraft.

Decodificación canónica. Siguiendo a Moffat y Turpin [21], el decodificador no reconstruye el árbol: lee un bit por iteración y, para cada longitud ℓ, comprueba si el valor acumulado cae en el rango de los c_ℓ códigos de esa longitud (algoritmo 2). Los símbolos se almacenan ordenados por código canónico, de modo que el índice del símbolo se obtiene por simple resta.

Algoritmo 2 Decodificación canónica de un símbolo (`decodeSymbol`).

Entrada: lector de bits; c_ℓ por longitud; símbolos ordenados por código canónico
1: $code \leftarrow 0$; $first \leftarrow 0$; $index \leftarrow 0$
2: **para** $\ell = 1$ **hasta** $L_{\text{máx}}$ **hacer**
3: $code \leftarrow code \,|\, \textsc{LeerBit}()$
4: **si** $code - first < c_\ell$ **entonces**
5: **devolver** $símbolos[index + code - first]$
6: **fin si**
7: $index \leftarrow index + c_\ell$; $first \leftarrow (first + c_\ell) \cdot 2$; $code \leftarrow code \cdot 2$
8: **fin para**
9: **error**: código inválido

Análisis de complejidad. Sea n el tamaño de la entrada y $\sigma \leq 256$ el número de símbolos presentes.

- *Conteo de frecuencias:* una pasada con un incremento por byte, $\Theta(n)$.
- *Construcción del código:* σ inserciones iniciales y $\sigma - 1$ iteraciones con dos extracciones y una inserción en el montículo, cada una $O(\log \sigma)$; en total $O(\sigma \log \sigma)$. La asignación de profundidades y de códigos canónicos cuesta $O(\sigma + L_{\text{máx}})$.
- *Codificación:* segunda pasada sobre la entrada; cada símbolo escribe ℓ_s bits con `BitWriter` en tiempo $O(1 + \ell_s/8)$, de modo que el total es $\Theta(n)$. El costo global de codificar es $\Theta(n + \sigma \log \sigma)$, es decir, $\Theta(n)$ para un alfabeto fijo.
- *Decodificación:* el algoritmo 2 realiza ℓ_s iteraciones por símbolo, por lo que el total es $\sum_i \ell_{x_i} = n\bar{L}$ iteraciones, es decir, $\Theta(n\bar{L})$. Por (4), $\bar{L} < H_0 + 1 \leq \log_2 256 + 1 = 9$, así que el costo es lineal en n, pero con una constante proporcional a la longitud media del código y, por tanto, a la entropía de la fuente.
- *Espacio adicional:* $O(\sigma)$ para el montículo, los pesos, los padres y las tablas canónicas, sin contar los arreglos de entrada y salida. La cabecera ocupa $2 + 4 + 256 = 262$ bytes, un sobrecosto de $8 \cdot 262/n$ bpb que solo es apreciable en entradas pequeñas.

3.3. Algoritmo Lempel–Ziv–Welch (LZW)

Idea. LZW [5] es una variante de LZ78 [6] que inicializa el diccionario con los 256 bytes posibles. El codificador extiende la frase actual w mientras $w\,c$ siga en el diccionario; cuando no lo está, emite el código de w, agrega $w\,c$ como nueva entrada y reinicia la frase con c. El decodificador reconstruye el mismo diccionario a partir de los códigos, sin que este se transmita. En esta implementación el código 256 se reserva para `CLEAR`, de modo que las frases nuevas comienzan en 257.

Códigos de ancho variable y reinicio. Los códigos se escriben con el menor ancho capaz de representar todos los códigos posibles en ese momento, $ancho(s) = \max(9, \lceil \log_2 s \rceil)$, donde s es el siguiente código libre; el ancho crece de 9 hasta b bits, con $b \in [9, 16]$ configurable (`maxBits`, 16 por defecto). Cuando el diccionario alcanza 2^b entradas, el codificador emite `CLEAR` y ambos lados vuelven al diccionario inicial. Esta política de reinicio mantiene el modelo adaptativo ante cambios en los datos y acota la memoria, a costa de olvidar frases útiles [26]. Los algoritmos 3 y 4 resumen `LzwCodec.compress` y `LzwCodec.decode`.

Algoritmo 3 Codificación LZW con reinicio (`compress`).

Entrada: bytes $x_1 \cdots x_n$ ($n \geq 1$), ancho máximo b

$D \leftarrow$ diccionario con los 256 bytes; $s \leftarrow 257$; $w \leftarrow x_1$
para $i = 2$ **hasta** n **hacer**
  $c \leftarrow x_i$
  si $(w, c) \in D$ **entonces**
    $w \leftarrow D[w, c]$ ▷ extender la frase
  si no
    emitir w con $\mathit{ancho}(s)$ bits
    si $s < 2^b$ **entonces**
      $D[w, c] \leftarrow s$; $s \leftarrow s + 1$
    si no
      emitir `CLEAR` con $\mathit{ancho}(s)$ bits; vaciar D; $s \leftarrow 257$
    fin si
    $w \leftarrow c$
  fin si
fin para
emitir w con $\mathit{ancho}(s)$ bits

Algoritmo 4 Decodificación LZW (`decode`).

Entrada: flujo de códigos, longitud original n, ancho máximo b

$s \leftarrow 257$; $\mathit{prev} \leftarrow \bot$
mientras se hayan producido menos de n bytes **hacer**
  leer k con $\mathit{ancho}(s)$ bits si $\mathit{prev} = \bot$, o con $\mathit{ancho}(\min(s + 1, 2^b))$ en otro caso
  si $k =$ `CLEAR` **entonces**
    $s \leftarrow 257$; $\mathit{prev} \leftarrow \bot$; **continuar**
  fin si
  si $\mathit{prev} = \bot$ **entonces**
    producir el byte k; $\mathit{prev} \leftarrow k$; **continuar**
  fin si
  si $s < 2^b$ **entonces**
    $h \leftarrow \mathit{primero}(k)$ si $k < s$; si no, $h \leftarrow \mathit{primero}(\mathit{prev})$ ▷ caso $KwKwK$
    $D[s] \leftarrow (\mathit{prev}, h)$; $s \leftarrow s + 1$
  fin si
  producir la frase $D[k]$ recorriendo prefijos de atrás hacia adelante; $\mathit{prev} \leftarrow k$
fin mientras

El decodificador agrega cada entrada un código más tarde que el codificador, porque necesita el primer byte de la frase siguiente; por eso lee con el ancho correspondiente a $s + 1$.

Ejemplo: `TOBEORNOTTOBEORTOBEORNOT`. La tabla 3 reproduce el ejemplo clásico de Welch con la numeración de esta implementación. Las 24 letras se codifican con 16 códigos, la secuencia `T O B E O R N O T 257 259 261 266 260 262 264`, que es exactamente la que verifica la prueba `emitsTextbookCodeSequence`. Como el diccionario nunca supera 512 entradas, todos los códigos usan 9 bits: $16 \cdot 9 = 144$ bits frente a $24 \cdot 8 = 192$ bits de entrada.

Tabla 3: Codificación LZW de `TOBEORNOTTOBEORTOBEORNOT` (solo los pasos que emiten).

Paso	Frase w	Siguiente c	Código emitido	Nueva entrada
1	`T`	`O`	84 (`T`)	257 = `TO`
2	`O`	`B`	79 (`O`)	258 = `OB`
3	`B`	`E`	66 (`B`)	259 = `BE`
4	`E`	`O`	69 (`E`)	260 = `EO`
5	`O`	`R`	79 (`O`)	261 = `OR`
6	`R`	`N`	82 (`R`)	262 = `RN`
7	`N`	`O`	78 (`N`)	263 = `NO`
8	`O`	`T`	79 (`O`)	264 = `OT`
9	`T`	`T`	84 (`T`)	265 = `TT`
10	`TO`	`B`	257	266 = `TOB`
11	`BE`	`O`	259	267 = `BEO`
12	`OR`	`T`	261	268 = `ORT`
13	`TOB`	`E`	266	269 = `TOBE`
14	`EO`	`R`	260	270 = `EOR`
15	`RN`	`O`	262	271 = `RNO`
16	`OT`	(fin)	264	—

El caso especial $KwKwK$. Si la entrada contiene $K\,w\,K\,w\,K$, con K un byte y w una cadena, el codificador puede emitir un código que acaba de crear y que el decodificador aún no conoce. Con la entrada `aaa`, por ejemplo, el codificador emite 97 (`a`), agrega 257 = `aa` y luego emite 257. Al recibir 257, el decodificador tiene $s = 257$: el código es igual al siguiente libre. La única frase consistente es la anterior seguida de su propio primer byte, `a` + `a` = `aa`, que es lo que hace la rama $k = s$ del algoritmo 4; la prueba `handlesCodeNotYetInDictionary` cubre este caso.

Análisis de complejidad. Sea $D = 2^b$ el tamaño máximo del diccionario.

- *Codificación:* el diccionario se implementa como una tabla hash de direccionamiento abierto con sondeo lineal y hashing de Fibonacci (multiplicación por `0x9E3779B1`), indexada por la clave (w, c). Su capacidad es $2D$, de modo que el factor de carga nunca supera $\alpha = 0.5$. Cada byte de entrada provoca una búsqueda; bajo la hipótesis de hashing uniforme, una búsqueda fallida requiere a lo sumo $1/(1-\alpha) = 2$ sondeos esperados [28]. Cada código emitido cuesta $O(1)$ adicional. Un reinicio limpia la tabla en $O(D)$, pero ocurre como mucho una vez cada $D - 257$ códigos, y cada código consume al menos un byte, de modo que su costo amortizado es $O(1)$ por byte. En total, $\Theta(n)$ esperado.
- *Decodificación:* cada código se resuelve con accesos a arreglos (`prefix`, `suffix`, `firstByte`, `phraseLength`) y cada byte de salida se escribe exactamente una vez al recorrer la frase hacia atrás; el costo es $\Theta(n)$ en el peor caso, sin dependencia de una función hash.
- *Espacio adicional:* $O(D)$, independiente de n. Para $b = 16$, el codificador usa dos arreglos de 2^{17} enteros y el decodificador cuatro arreglos de 2^{16} entradas. La cabecera ocupa solo 7 bytes (magia, b y longitud original).

3.4. Comparación teórica

La tabla 4 resume las diferencias estructurales entre ambos algoritmos tal como se implementaron. La diferencia esencial no es asintótica —ambos son lineales— sino de *modelo*: Huffman es estático y de dos pasadas, y explota solo la distribución de frecuencias de los bytes; LZW es adaptativo y de una pasada, y explota frases repetidas, pero paga al menos 9 bits por código emitido.

Tabla 4: Comparación teórica de las implementaciones.

Aspecto	Huffman canónico	LZW (9–b bits, `CLEAR`)
Tiempo de codificación	$\Theta(n + \sigma \log \sigma) = \Theta(n)$	$\Theta(n)$ esperado (tabla hash)
Tiempo de decodificación	$\Theta(n\bar{L})$, con $\bar{L} < H_0 + 1 \leq 9$	$\Theta(n)$ en el peor caso
Espacio adicional	$O(\sigma)$	$O(2^b)$, independiente de n
Modelo	Estático, dos pasadas; requiere transmitir el código	Adaptativo, una pasada; el diccionario no se transmite
Redundancia explotada	Frecuencias de símbolos (orden cero)	Frases repetidas
Garantía de tasa	$H_0 \leq \bar{L} < H_0 + 1$	Asintótica para fuentes estacionarias [6]; sin cota para n finito
Cabecera	262 bytes	7 bytes

3.5. Hipótesis

Del análisis anterior se derivan cuatro hipótesis que guían los experimentos de la sección 4:

H1. Los codificadores y decodificadores de ambos algoritmos escalan linealmente con el tamaño de la entrada: la pendiente del ajuste log–log del tiempo frente a n es cercana a 1.

H2. Los bits por byte de Huffman se ubican en $[H_0, H_0 + 1)$ y, en fuentes i.i.d., quedan cerca de H_0.

H3. LZW supera a Huffman en datos con frases repetidas (texto natural y sintético), pierde frente a él en datos i.i.d. y expande los datos aleatorios uniformes (bpb > 8).

H4. El tiempo de decodificación de Huffman crece con la longitud media del código (aproximadamente, con la entropía), mientras que el de LZW es prácticamente insensible a ella.

4. Metodología

4.1. Proceso de comparación

La figura 2 resume los pasos de la comparación. Se parte de conjuntos de datos elegidos para aislar cada tipo de redundancia; ambos algoritmos se implementan detrás de una misma interfaz y se validan antes de medir; cada configuración se mide tras calentar el compilador JIT, con repeticiones cronometradas y resúmenes estadísticos robustos; finalmente, la tasa de crecimiento estimada y la tasa de compresión se contrastan con las hipótesis H1–H4 derivadas del análisis teórico. El diseño sigue las pautas de la algoritmia experimental [27, 15] y de la evaluación de rendimiento en Java [14].

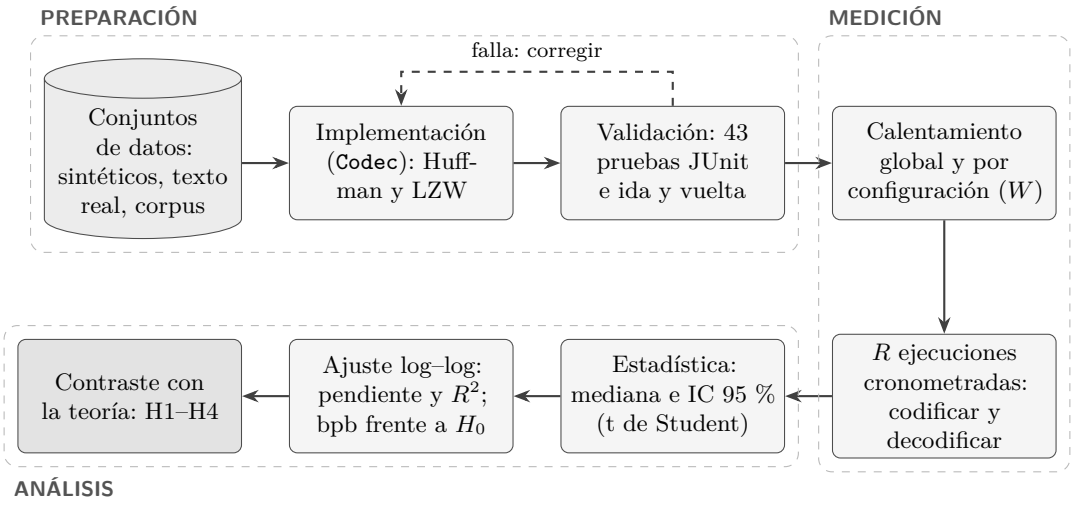


Figura 2: Metodología de la comparación entre Huffman y LZW.

4.2. Implementación

Arquitectura. El código es un proyecto NetBeans de tipo *Java with Ant* compilado para Java 17 y ejecutado con el JDK 21. Todos los algoritmos implementan la interfaz `Codec` (métodos `name`, `encode` y `decode`), que exige que `decode(encode(x))` sea igual a `x` y que los datos corruptos se rechacen con `IllegalArgumentException`. Gracias a esa interfaz, el arnés de medición trata por igual a Huffman y a LZW. La tabla 5 describe los paquetes.

Tabla 5: Paquetes del proyecto (`compression`).

Paquete	Clases	Responsabilidad
(raíz)	`Codec`	Interfaz común de los algoritmos.
`io`	`BitWriter`, `BitReader`	E/S de campos de bits de ancho variable, bit más significativo primero, con acumulador de 64 bits.
`huffman`	`HuffmanCodec`	Longitudes con montículo, códigos canónicos y decodificación canónica.
`lzw`	`LzwCodec`	Códigos de 9 a 16 bits, diccionario con tabla hash y reinicio con `CLEAR`.
`bench`	`BenchmarkRunner`, `Stats`, `DataGenerator`, `ResultWriter`	Experimentos E1–E3, estadística, generación de datos y escritura de CSV y macros LaTeX.
`app`	`Main`	Interfaz de línea de comandos (`compress`, `decompress`, `stats`, `demo`).

Núcleo de Huffman. El listado 1 muestra el bucle principal de `HuffmanCodec.codeLengths`. Los nodos internos reciben índices crecientes a partir de 256, de modo que un padre siempre tiene un índice mayor que sus hijos y basta un recorrido descendente para asignar profundidades sin recursión. El montículo es un `PriorityQueue` que desempata por índice, lo que hace el código determinista.

```
int next = ALPHABET; // los nodos internos reciben los índices 256, 257, ...
while (heap.size() > 1) {
    int a = heap.poll();
    int b = heap.poll();
    weight[next] = weight[a] + weight[b];
    parent[a] = next;
    parent[b] = next;
    heap.add(next++);
```

```
}

// Un padre siempre tiene un índice mayor que sus hijos: basta un recorrido descendente.
int root = next - 1;
int[] depth = new int[maxNodes];
for (int node = root - 1; node >= ALPHABET; node--) {
    depth[node] = depth[parent[node]] + 1;
}
```

Listado 1: Fusión de nodos y cálculo de profundidades en `HuffmanCodec.codeLengths`.

Núcleo de LZW. El listado 2 reproduce `LzwCodec.compress`. La clave de la tabla hash concatena el código de la frase y el byte siguiente en un entero; una búsqueda exitosa extiende la frase, y una fallida emite el código y deja preparada la ranura (`slot`) donde se inserta la nueva entrada, sin una segunda búsqueda. El destino de los códigos (`CodeSink`) es el `BitWriter` al comprimir, o un arreglo al trazar los códigos en las pruebas.

```
private void compress(byte[] input, CodeSink sink) {
    if (input.length == 0) {
        return;
    }
    int maxSize = 1 << maxBits;
    PhraseTable table = new PhraseTable(maxBits);
    int size = FIRST_FREE;
    int phrase = input[0] & 0xFF;
    for (int i = 1; i < input.length; i++) {
        int next = input[i] & 0xFF;
        int key = (phrase << 8) | next;
        int slot = table.find(key);
        if (table.isOccupied(slot)) {
            phrase = table.valueAt(slot); // extender la frase actual
            continue;
        }
        sink.put(phrase, width(size));
        if (size < maxSize) {
            table.insert(slot, key, size++);
        } else {
            sink.put(CLEAR, width(size));
            table.clear();
            size = FIRST_FREE;
        }
        phrase = next;
    }
    sink.put(phrase, width(size));
}
```

Listado 2: Bucle de codificación de `LzwCodec.compress`.

4.3. Validación de la corrección

Antes de medir, la corrección se verifica en dos niveles. El primero es una batería de 43 pruebas JUnit 4 que se ejecuta con `ant test`. El segundo está incorporado en el arnés: cada configuración medida se codifica y decodifica una vez y, si el resultado no es idéntico a la entrada, la campaña se detiene con un error. Así, ningún tiempo reportado corresponde a una ejecución incorrecta. La tabla 6 resume la batería por clase.

Tabla 6: Batería de pruebas JUnit (43 pruebas).

Clase de prueba	Pruebas	Aspectos verificados
`HuffmanCodecTest`	11	Ida y vuelta (vacío, un byte, un solo símbolo repetido con 1 bit por símbolo, los 256 valores, datos aleatorios y sesgados, texto); longitudes óptimas del ejemplo de libro; igualdad de Kraft; códigos canónicos libres de prefijo y ordenados por longitud; rechazo de entradas corruptas; nombre del algoritmo.
`BitStreamTest`	5	Orden MSB primero y relleno con ceros; anchos aleatorios hasta 64 bits; desplazamiento inicial; lectura más allá del final; anchos inválidos.
`LzwCodecTest`	8	Secuencia del ejemplo de Welch; vacío y un byte; caso $KwKwK$; todos los anchos de 9 a 16 bits con reinicios del diccionario; los 256 valores; compresión de datos repetitivos; configuración inválida y datos corruptos; ancho registrado en la cabecera.
`StatsTest`	6	Resumen de muestras, mediana con cantidad par, cuantil normal para muestras grandes, rechazo de muestras insuficientes, exponente de la ley de potencia en escala log–log, entropía de orden cero.
`DataGeneratorTest`	3	Determinismo por semilla, entropía esperada de cada fuente, uso exacto de k símbolos.
`BenchmarkRunnerTest`	4	Una corrida rápida escribe todos los archivos de resultados; `--only e2` escribe solo el barrido; argumentos desconocidos o incompletos.
`MainTest`	6	Compresión y descompresión de archivos con ambos algoritmos; comando `stats`; algoritmo y comando desconocidos (código de salida 2); archivo corrupto (código 1); demostración sin argumentos.

4.4. Conjuntos de datos

Para separar el efecto del algoritmo del efecto del tipo de redundancia se combinan fuentes sintéticas, cuyas propiedades se conocen por construcción, con texto real y con los corpus estándar de evaluación de compresores (tabla 7). Las fuentes sintéticas se generan con `java.util.Random` y semilla fija 2026, por lo que son reproducibles.

Tabla 7: Conjuntos de datos.

Conjunto	Construcción	Redundancia que contiene	Uso
`uniform`	Bytes i.i.d. uniformes en $\{0, \ldots, 255\}$	Ninguna ($H_0 \approx 8$)	E1
`skewed`	Símbolos i.i.d. geométricos con $p = 0.15$, truncados a 255	Frecuencias sesgadas, sin frases	E1
`synthtext`	Palabras de un vocabulario de 2000 (2 a 9 letras con frecuencias del inglés) elegidas con ley de Zipf de exponente 1.1, separadas por espacios y saltos de línea	Frases repetidas y frecuencias sesgadas	E1
`dickens`	Prefijos del archivo `dickens` de Silesia (texto en inglés)	Texto natural	E1
Alfabetos k	Símbolos i.i.d. uniformes en $\{0, \ldots, k-1\}$, $k = 2, 4, \ldots, 256$, $n = 2^{23}$	Entropía controlada $H_0 = \log_2 k$	E2
Canterbury [16]	11 archivos del corpus estándar y 3 del corpus grande (`E.coli`, `bible.txt`, `world192.txt`)	Texto, código, binarios, ADN	E3
Silesia [17]	12 archivos de tipos diversos (texto, XML, bases de datos, ejecutables, imágenes médicas)	Mezcla realista	E3

Los corpus se descargan con `data/download-corpora.sh`, que verifica sumas SHA-256 antes de extraerlos.

4.5. Métricas

- *Tiempo:* mediana, en milisegundos, de R ejecuciones de codificación y de decodificación, medidas por separado con `System.nanoTime`.
- *Dispersión:* semiancho del intervalo de confianza del 95 % de la media, $t_{0.975,\,R-1}\, s/\sqrt{R}$, con la tabla de cuantiles de Student para hasta 30 grados de libertad y 1.96 por encima.
- *Costo por byte:* ns/byte $= 10^6 \cdot$ mediana [ms]$/n$; a complejidad lineal, esta métrica debe ser aproximadamente constante en n. En el corpus se reporta el rendimiento en MB/s, $n/10^6$ dividido entre la mediana en segundos.
- *Compresión:* bits por byte según la ecuación (3), incluida la cabecera, y su comparación con H_0 de la ecuación (1).
- *Tasa de crecimiento:* pendiente $\hat{k}$ y R^2 del ajuste por mínimos cuadrados de $\ln t = \hat{k} \ln n + c$; para un algoritmo $\Theta(n^k)$ la pendiente tiende a k [15]. El ajuste usa los tamaños $n \geq 2^{16}$, para que los costos fijos no dominen.

4.6. Protocolo de medición

El protocolo implementado en `BenchmarkRunner` responde a los riesgos que Georges et al. [14] identifican en la medición de programas Java —compilación JIT, recolección de basura y eliminación de código muerto— y a las recomendaciones de reproducibilidad de Johnson [27] y McGeoch [15]:

1. *Calentamiento global:* antes del primer experimento, cada códec (Huffman y LZW de 16 bits) codifica y decodifica tres veces 4 MiB de cada fuente sintética, para que los primeros puntos no se midan en el intérprete.
2. *Preparación de cada configuración:* se invoca `System.gc()`, se codifica una vez, se registra el tamaño comprimido y se verifica la ida y vuelta.

3. *Calentamiento local y repeticiones:* se ejecutan W iteraciones no medidas y luego R iteraciones medidas, según el tamaño: $W = 5$, $R = 21$ para $n \leq 1$ MiB; $W = 3$, $R = 11$ para $n \leq 16$ MiB; y $W = 1$, $R = 7$ por encima.
4. *Sumidero:* las salidas se acumulan en un campo `volatile` (`blackhole`) para impedir que el JIT elimine el trabajo medido.
5. *Reproducibilidad:* semilla fija, parámetros de la JVM fijos y escritura automática de los resultados en CSV y en macros LaTeX que este informe importa sin transcripción manual. Los parámetros de la JVM son `-Xms3g`, `-Xmx3g` y `-XX:+UseParallelGC`.

4.7. Diseño experimental

La tabla 8 relaciona cada experimento con la variable que manipula, los parámetros fijos, las salidas medidas y la hipótesis que contrasta.

Tabla 8: Experimentos.

Exp.	Variable	Parámetros fijos	Salidas	Hipótesis
E1	$n = 2^{14}, \ldots, 2^{24}$ (`dickens`: hasta 2^{23})	LZW con $b = 16$; fuentes `uniform`, `skewed`, `synthtext`, `dickens`	Tiempos (mediana, IC), ns/byte, bpb, pendiente y R^2	H1 (y H2, H3)
E2	$k = 2, 4, \ldots, 256$ ($H_0 = \log_2 k$)	$n = 2^{23}$; fuente i.i.d. uniforme en k símbolos; LZW con $b = 16$	bpb, tiempos de codificación y decodificación	H2, H4 (y H3)
E3	Archivo del corpus	Canterbury y Silesia completos; LZW con $b = 16$	bpb, MB/s, archivos donde gana LZW	H3 (y H2)

4.8. Entorno de ejecución

Todas las mediciones se realizaron en una misma máquina, cuyos datos registra automáticamente el arnés (tabla 9).

Tabla 9: Entorno de ejecución (generado por `BenchmarkRunner`).

Elemento	Valor
Procesador	Intel(R) Core(TM) Ultra 7 265K
Núcleos lógicos disponibles	20
Máquina virtual Java	OpenJDK 64-Bit Server VM 21.0.13
Memoria máxima del montículo	2944 MiB
Opciones de la JVM	`-Xms3g -Xmx3g -XX:+UseParallelGC`
Compilación	Java 17 (`javac.source`), Apache Ant, JUnit 4.13.2
Fecha de la campaña	2026-11-09

4.9. Amenazas a la validez

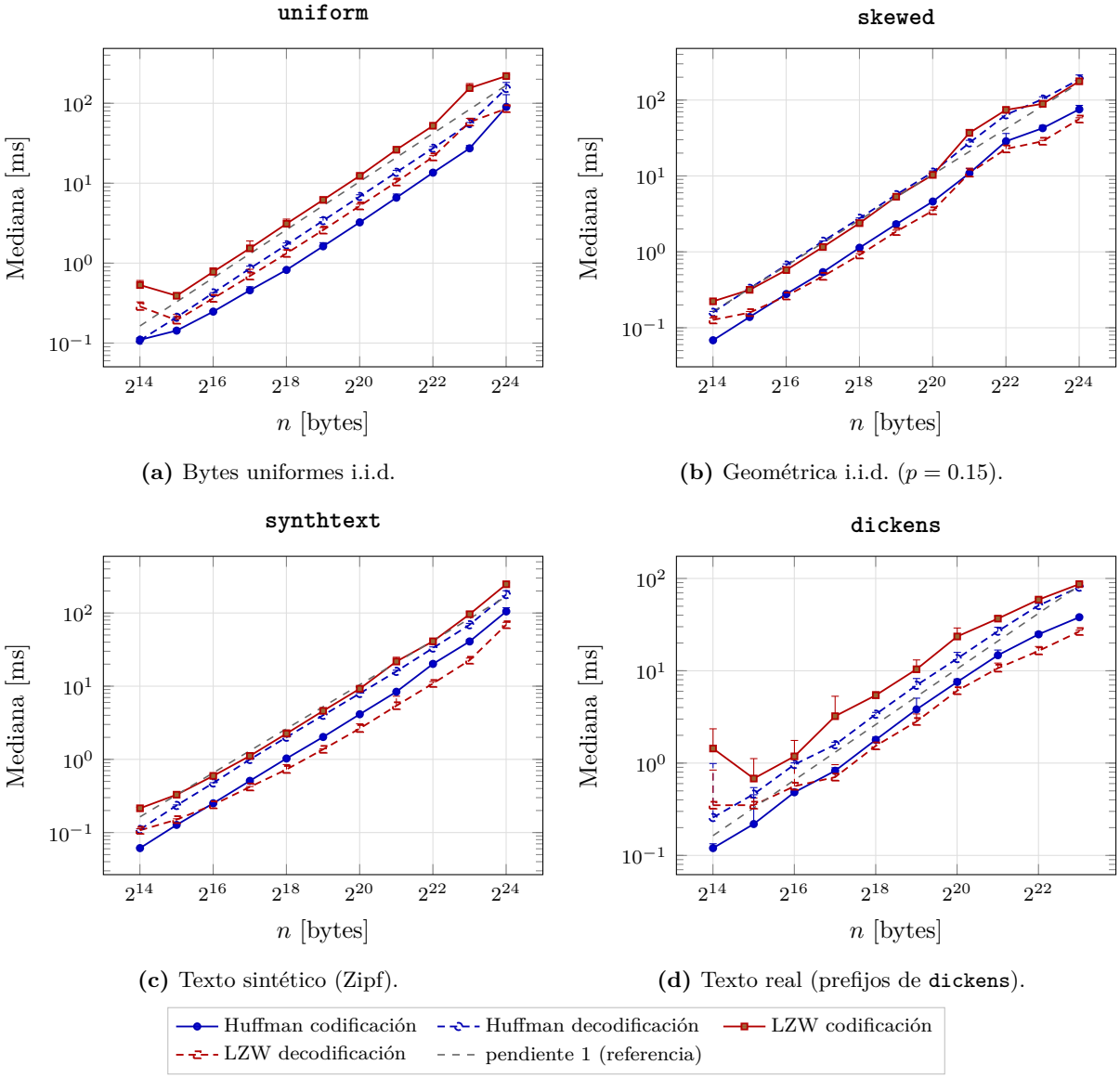
- *Ruido de JIT y GC.* Aunque el calentamiento, las repeticiones y la mediana mitigan la variabilidad [14], las pausas del recolector o la recompilación pueden afectar mediciones individuales; los intervalos de confianza lo hacen visible.
- *Una sola máquina.* Los valores absolutos dependen del procesador, la jerarquía de caché y la JVM; las conclusiones se basan en tendencias (pendientes, ns/byte relativos) más que en tiempos absolutos.

- *Datos sintéticos frente a reales.* Las fuentes sintéticas aíslan propiedades, pero no representan todos los datos reales; por eso se complementan con texto real y con los corpus Canterbury y Silesia.
- *Calidad de la implementación.* Los tiempos reflejan estas implementaciones concretas (por ejemplo, la decodificación de Huffman bit a bit, sin tablas de búsqueda), no el límite de cada algoritmo; las optimizaciones conocidas se discuten en la literatura [21, 22].

5. Resultados

5.1. E1: escalamiento del tiempo con el tamaño de la entrada

La figura 3 muestra la mediana del tiempo de codificación y de decodificación en función de n para las cuatro fuentes, en escala log–log; la recta discontinua tiene pendiente 1 y sirve solo como guía visual de crecimiento lineal (su posición vertical es arbitraria). Las barras de error indican el semiancho superior del intervalo de confianza del 95 %.



(a) Bytes uniformes i.i.d. **(b)** Geométrica i.i.d. ($p = 0.15$).

(c) Texto sintético (Zipf). **(d)** Texto real (prefijos de `dickens`).

Figura 3: E1: mediana del tiempo frente al tamaño de la entrada (escala log–log).

En las cuatro fuentes, las curvas de las cuatro operaciones son aproximadamente paralelas a

la recta de pendiente 1 a partir de $n \approx 2^{16}$: duplicar la entrada duplica, en buena aproximación, el tiempo. En los tamaños menores la tendencia se rompe y los intervalos se ensanchan, porque los costos fijos de cada llamada —construir el código y escribir la cabecera de Huffman, reservar e inicializar las tablas de LZW— son comparables con el trabajo proporcional a n, y porque mediciones de décimas de milisegundo son más sensibles al ruido del JIT y del recolector. El caso extremo es la codificación LZW de `dickens`: 1.44 ms con 16 KiB frente a 0.68 ms con 32 KiB (`e1_dickens.csv`), con un semiancho de intervalo de 0.91 ms. El orden entre operaciones es estable: la decodificación de LZW es la operación más rápida en todas las fuentes; la codificación de LZW es la más lenta en tres de ellas, y en `skewed` la supera ligeramente la decodificación de Huffman; y, en Huffman, decodificar es siempre más lento que codificar.

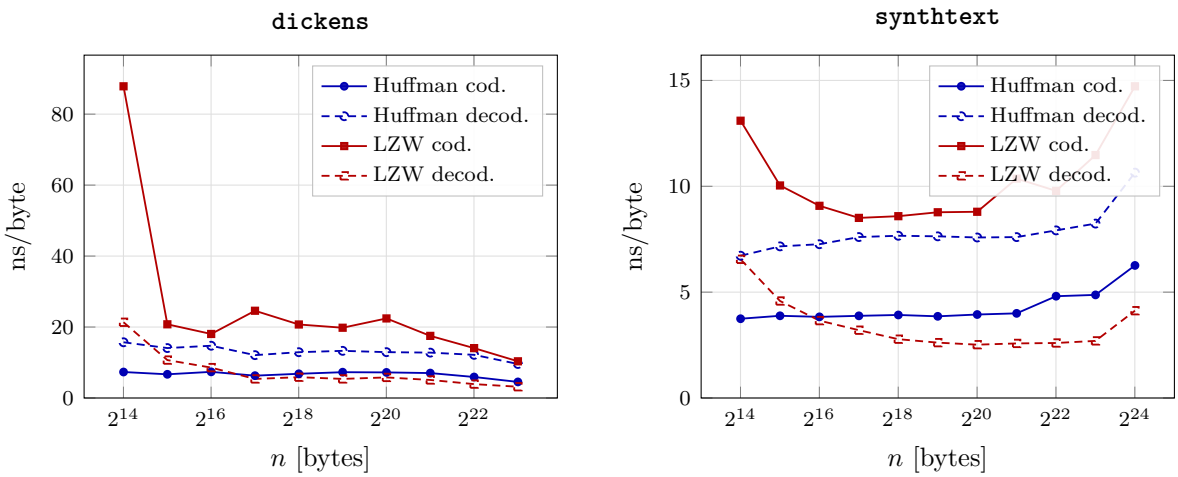
La tabla 10 reporta la pendiente del ajuste log–log y su R^2 para cada fuente, algoritmo y operación, calculados con $n \geq 2^{16}$, junto con el costo por byte en el mayor tamaño medido. Las dos últimas columnas corresponden a una réplica independiente de E1, ejecutada con el mismo protocolo en la misma máquina, que permite estimar la variabilidad entre corridas.

Tabla 10: E1: pendiente $\hat{k}$ del ajuste $\ln t = \hat{k} \ln n + c$, coeficiente R^2 y costo por byte en el mayor n, en la corrida principal y en la réplica.

			Corrida principal			Réplica	
Fuente	Algoritmo	Operación	$\hat{k}$	R^2	ns/byte en $n_{\text{máx}}$	$\hat{k}$	R^2
`uniform`	Huffman	codificación	1.031	0.9927	5.37	0.984	0.9987
`uniform`	Huffman	decodificación	1.035	0.9978	9.14	1.009	0.9999
`uniform`	LZW	codificación	1.046	0.9966	13.05	1.024	0.9985
`uniform`	LZW	decodificación	1.013	0.9970	5.11	1.005	0.9977
`skewed`	Huffman	codificación	1.047	0.9957	4.50	1.010	0.9998
`skewed`	Huffman	decodificación	1.042	0.9972	11.18	1.013	1.0000
`skewed`	LZW	codificación	1.076	0.9871	10.54	1.024	0.9998
`skewed`	LZW	decodificación	1.009	0.9905	3.35	0.972	0.9992
`synthtext`	Huffman	codificación	1.074	0.9980	6.26	1.005	1.0000
`synthtext`	Huffman	decodificación	1.044	0.9985	10.64	1.006	1.0000
`synthtext`	LZW	codificación	1.078	0.9975	14.72	1.007	0.9999
`synthtext`	LZW	decodificación	0.996	0.9913	4.12	0.931	0.9979
`dickens`	Huffman	codificación	0.938	0.9939	4.53	0.997	0.9999
`dickens`	Huffman	decodificación	0.948	0.9972	9.59	0.995	0.9999
`dickens`	LZW	codificación	0.879	0.9845	10.34	1.006	0.9999
`dickens`	LZW	decodificación	0.847	0.9909	3.17	0.935	0.9982

En la corrida principal, las pendientes de las fuentes sintéticas están entre 1.00 y 1.08, con $R^2 \geq 0.987$; en `dickens` son algo menores que 1: Huffman 0.94 (codificación) y 0.95 (decodificación), LZW 0.88 y 0.85, con $R^2 \geq 0.985$. En la réplica, las dieciséis pendientes quedan entre 0.93 y 1.02, con $R^2 \geq 0.997$ en todas las series; en `dickens`, entre 0.93 y 1.01. La mayor diferencia entre corridas para una misma serie es de 0.13 (codificación LZW de `dickens`: 0.879 frente a 1.006), lo que indica que la segunda cifra decimal de la pendiente está dentro de la variabilidad experimental. En ningún caso la pendiente se acerca a 2, como correspondería a un crecimiento cuadrático.

La figura 4 presenta el mismo experimento como costo por byte en escala semilogarítmica: bajo complejidad lineal, las curvas deben ser aproximadamente horizontales.

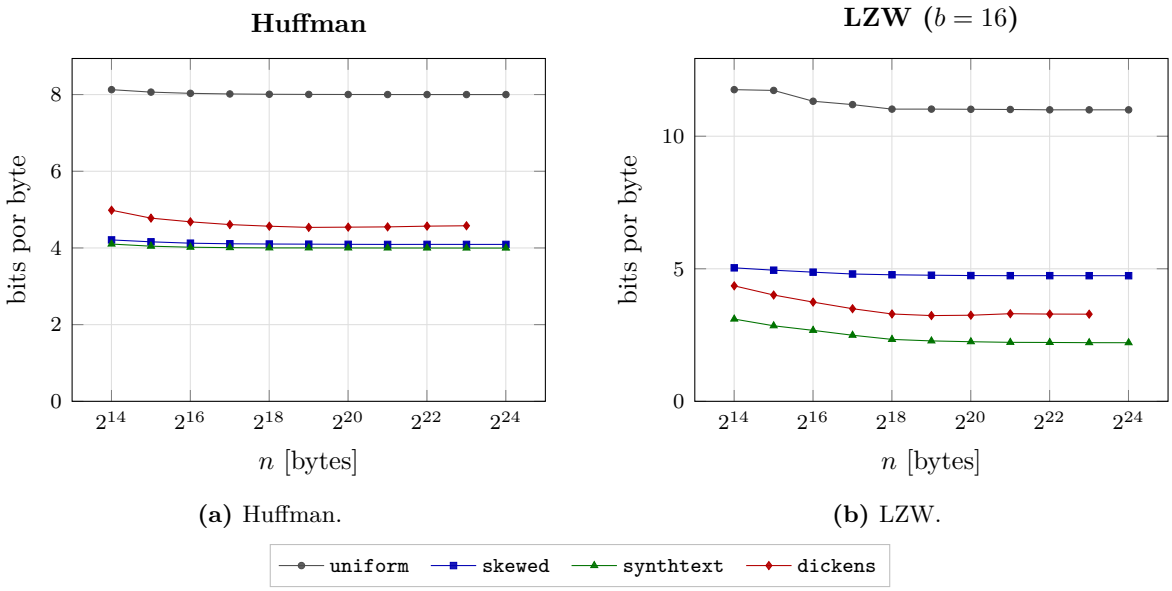


(a) Texto real. **(b)** Texto sintético.

Figura 4: E1: costo por byte (ns/byte) frente al tamaño de la entrada.

El costo por byte disminuye con n en los primeros tamaños y luego se estabiliza, que es la firma de un costo fijo por llamada amortizado sobre más bytes. En `dickens`, la codificación LZW pasa de 87.85 ns/byte con 16 KiB a 10.34 ns/byte con 8 MiB, y la decodificación de Huffman de 15.71 a 9.59 ns/byte. En `synthtext`, la codificación de Huffman se mantiene entre 3.83 y 4.81 ns/byte desde 2^{16} hasta 2^{22} bytes y solo sube en los dos mayores tamaños (4.87 y 6.26 ns/byte), comportamiento compatible con datos y salidas que ya no caben en la caché del procesador. En el mayor n de cada fuente (tabla 10), los costos se ordenan así: la decodificación de LZW cuesta entre 3.17 y 5.11 ns/byte, la codificación de Huffman entre 4.50 y 6.26, la decodificación de Huffman entre 9.14 y 11.18, y la codificación de LZW entre 10.34 y 14.72. Las asimetrías son opuestas: en Huffman decodificar cuesta entre 1.7 y 2.5 veces más que codificar, porque el decodificador canónico procesa un bit por iteración; en LZW, codificar cuesta entre 2.6 y 3.6 veces más que decodificar, porque el codificador realiza una búsqueda en la tabla hash por cada byte de entrada, mientras que el decodificador resuelve cada código con accesos directos a arreglos y copia la frase completa.

Finalmente, la figura 5 muestra la tasa de compresión obtenida en E1 para cada fuente y tamaño.



(a) Huffman. **(b)** LZW.

Figura 5: E1: bits por byte frente al tamaño de la entrada.

En Huffman, la tasa apenas depende de n: solo en los tamaños pequeños se nota la cabecera de 262 bytes, que con $n = 2^{14}$ añade $8 \cdot 262/16384 \approx 0.13$ bpb; así, `uniform` pasa de 8.13 bpb con 16 KiB a 8.0001 bpb con 16 MiB. En LZW, la tasa mejora a medida que el diccionario aprende frases más largas: `synthtext` baja de 3.10 a 2.21 bpb y `dickens` de 4.36 a 3.29 bpb, mientras que `skewed` solo baja de 5.04 a 4.74 bpb. En `uniform`, LZW expande los datos en todos los tamaños: incluso con 16 MiB produce 11.00 bpb, un 37.4 % más que la entrada sin comprimir.

5.2. E2: barrido de entropía

En E2 la fuente es i.i.d. uniforme sobre k símbolos, de modo que $H_0 = \log_2 k$ se conoce por construcción y no hay frases repetidas más allá de las debidas al azar. La figura 6 compara los bits por byte de ambos algoritmos con la entropía empírica y con la cota superior $H_0 + 1$ de la ecuación (4).

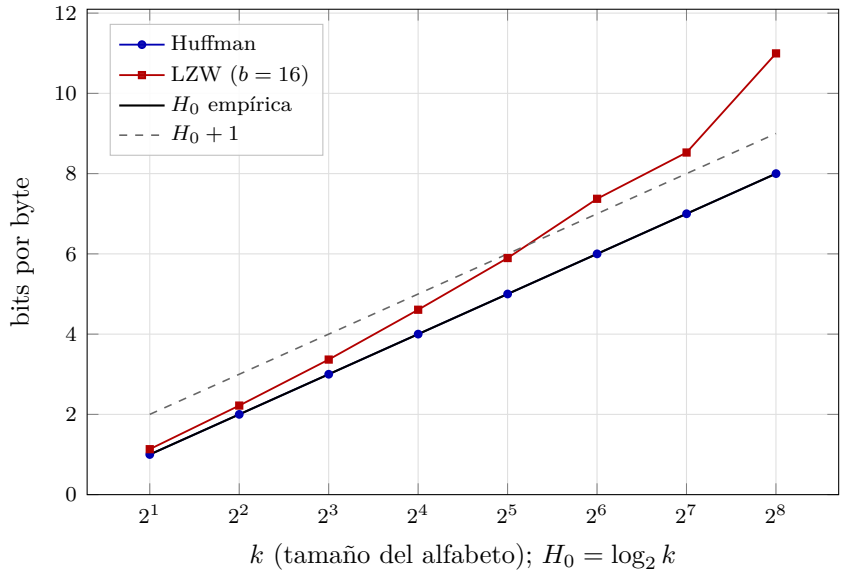
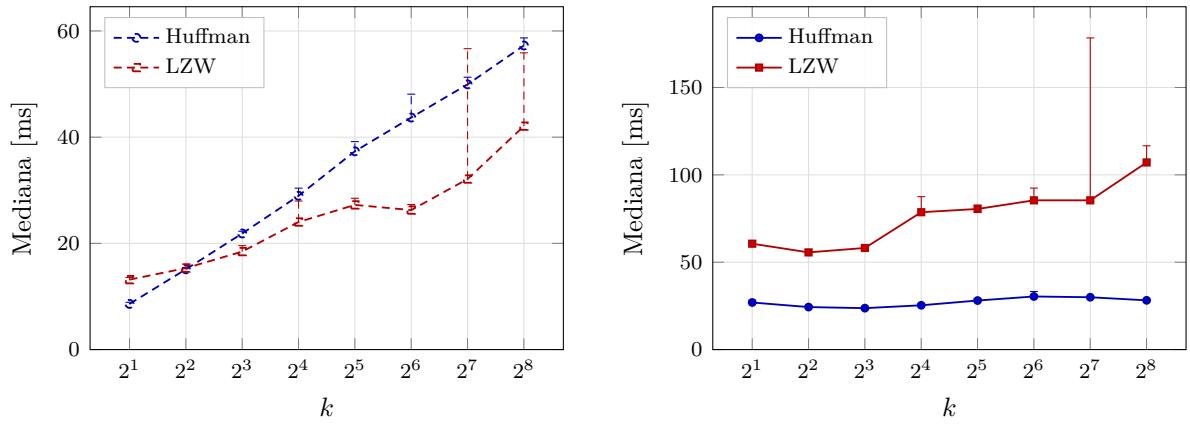


Figura 6: E2: bits por byte frente al tamaño del alfabeto ($n = 2^{23}$).

Huffman coincide con la entropía en todo el barrido: para cada k produce exactamente $H_0 + 0.0002$ bpb (de 1.0002 con $k = 2$ a 8.0002 con $k = 256$), y esa diferencia es la cabecera, $8 \cdot 262/2^{23} \approx 0.00025$ bpb. LZW, en cambio, queda por encima de H_0 para todo k y la distancia crece con el tamaño del alfabeto: 1.13 frente a 1.00 bpb con $k = 2$, 4.61 frente a 4.00 con $k = 16$ y 11.00 frente a 8.00 con $k = 256$. Desde $k = 64$ LZW supera incluso la cota $H_0 + 1$ de Huffman (7.37 bpb frente a 7), y con $k \geq 128$ expande los datos (8.53 y 11.00 bpb).

La figura 7 muestra cómo varía el tiempo con la entropía para un tamaño fijo, lo que permite contrastar el término $\bar{L}$ del costo de decodificación de Huffman.



(a) Decodificación. **(b)** Codificación.

Figura 7: E2: tiempo frente al tamaño del alfabeto ($n = 2^{23}$).

La decodificación de Huffman pasa de 8.60 ms con $k = 2$ a 57.26 ms con $k = 256$, un factor de 6.7. Como el eje horizontal es logarítmico, la recta casi perfecta de la figura 7a indica un crecimiento lineal en $\log_2 k$: cada bit adicional de longitud de código añade entre 6.25 y 8.37 ms, unos 6.95 ms en promedio, es decir, alrededor de 0.83 ns por bit decodificado. La decodificación de LZW también crece, de 13.17 a 42.08 ms (factor 3.2), pero con menor pendiente y con más dispersión: con $k = 128$ el semiancho del intervalo es de 24.55 ms en la decodificación y de 92.96 ms en la codificación, señal de iteraciones atípicas en esa configuración. La codificación de Huffman es casi insensible a la entropía (entre 23.71 y 30.39 ms), mientras que la de LZW sube de 60.60 a 107.11 ms.

5.3. E3: corpus Canterbury y Silesia

La tabla 11 reporta, para cada archivo, su tamaño, su entropía de orden cero, los bits por byte de cada algoritmo y el rendimiento de codificación. En negrita aparece el mejor de los dos algoritmos.

Tabla 11: E3: bits por byte y rendimiento de codificación en los corpus Canterbury (arriba) y Silesia (abajo).

Archivo	Tamaño (KiB)	H_0	Huffman	LZW	Huff. MB/s	LZW MB/s
`E.coli`	4530	2.00	**2.00**	2.17	348	141
`alice29.txt`	149	4.57	4.63	**3.27**	222	105
`asyoulik.txt`	122	4.81	4.86	**3.51**	228	107
`bible.txt`	3953	4.34	4.39	**2.78**	234	98
`cp.html`	24	5.23	5.35	**3.68**	231	89
`fields.c`	11	5.01	5.23	**3.56**	206	69
`grammar.lsp`	4	4.63	5.23	**3.91**	214	32
`kennedy.xls`	1006	3.57	3.60	**2.40**	286	142
`lcet10.txt`	417	4.67	4.70	**3.19**	235	102
`plrabn12.txt`	471	4.53	4.58	**3.41**	228	104
`ptt5`	501	1.21	1.66	**0.97**	251	203
`sum`	37	5.33	5.42	**4.21**	186	91
`world192.txt`	2415	5.00	5.04	**3.26**	196	96
`xargs.1`	4	4.90	5.42	**4.43**	91	5
`dickens`	9954	4.53	4.57	**3.29**	216	95
`mozilla`	50020	6.22	6.24	**4.30**	150	63
`mr`	9737	3.68	3.71	**3.01**	246	112
`nci`	32767	2.43	2.44	**0.94**	264	137
`ooffice`	6008	6.64	6.66	**5.29**	205	83
`osdb`	9849	6.59	6.62	**4.38**	219	84
`reymont`	6472	4.84	4.87	**2.46**	218	106
`samba`	21100	6.09	6.13	**3.06**	212	100
`sao`	7082	7.53	7.55	**7.53**	214	80
`webster`	40487	4.97	5.00	**2.76**	207	95
`x-ray`	8276	6.60	**6.63**	6.67	237	102
`xml`	5220	5.52	5.55	**1.66**	247	103

Las columnas Huffman y LZW están en bits por byte, incluida la cabecera.

En conjunto, sobre los 26 archivos, la tasa agregada es de 5.12 bpb para Huffman y 3.30 bpb para LZW; LZW comprime mejor que Huffman en 24 de los 26 archivos.

Huffman queda siempre por encima de H_0 y por debajo de $H_0 + 1$. En los archivos de más de 1 MB la distancia no supera 0.044 bpb (`world192.txt`); es mayor en los archivos muy pequeños, donde pesa la cabecera, y en `ptt5`, que se analiza en la sección 6.2. LZW gana en 24 de los 26 archivos y en 23 de ellos baja de H_0; las mayores ventajas aparecen en datos estructurados con frases recurrentes: `xml` (1.66 frente a 5.55 bpb), `nci` (0.94 frente a 2.44), `samba` (3.06 frente a 6.13) y `reymont` (2.46 frente a 4.87). Huffman gana en `E.coli` (2.00 frente a 2.17 bpb) y en `x-ray` (6.63 frente a 6.67), y `sao` es prácticamente un empate (7.55 frente a 7.53). Ningún archivo del corpus lleva a LZW por encima de 8 bpb.

En rendimiento, la codificación de Huffman procesa entre 150 y 348 MB/s en los archivos de más de 1 MB, y la de LZW entre 63 y 142 MB/s (203 MB/s en `ptt5`). En la decodificación el orden se invierte: LZW alcanza entre 194 y 621 MB/s en esos archivos y Huffman entre 79 y 558 MB/s, con los valores altos en archivos de entropía baja (`E.coli`, 558 MB/s; `ptt5`, 360 MB/s) y los bajos en archivos de entropía alta (`mozilla`, 79 MB/s; `sao`, 82 MB/s) (`e3_corpus.csv`). En los archivos diminutos el rendimiento de LZW se desploma —32 MB/s en `grammar.lsp` y 4.5 MB/s en `xargs.1`— porque cada llamada reserva e inicializa tablas de tamaño $O(2^b)$, independiente de n, cuyo costo domina cuando la entrada tiene unos pocos kilobytes; la diferencia entre ambos archivos, de tamaño similar, sugiere además ruido de medición a esa escala.

6. Discusión

Esta sección contrasta cada hipótesis de la sección 3.5 con la evidencia de la sección 5.

6.1. H1: escalamiento lineal

Predicción. Según la tabla 4, los cuatro procedimientos son lineales en n: la codificación de Huffman es $\Theta(n+\sigma \log \sigma)$, su decodificación $\Theta(n\bar{L})$ con $\bar{L}$ acotado, y ambas operaciones de LZW son $\Theta(n)$. En escala log–log, la pendiente esperada es 1.

Desviaciones previsibles. El análisis asintótico ignora costos fijos que pesan en entradas pequeñas: Huffman construye el código y escribe una cabecera de 262 bytes en cada llamada, y LZW reserva e inicializa en cada llamada su tabla hash de 2^{17} posiciones (dos arreglos de enteros) o, al decodificar, cuatro arreglos de 2^{16} entradas. Estos costos constantes aplanan la curva en los primeros tamaños y tienden a producir pendientes menores que 1 si se incluyen en el ajuste; por eso el ajuste usa $n \geq 2^{16}$. En el extremo opuesto, cuando los datos y las tablas dejan de caber en la memoria caché, el costo por byte puede aumentar ligeramente sin que cambie la complejidad.

Evidencia. Figuras 3 y 4 y tabla 10.

Contraste. H1 se acepta para las cuatro operaciones. Las pendientes de la corrida principal van de 1.00 a 1.08 en las fuentes sintéticas ($R^2 \geq 0.987$) y de 0.85 a 0.95 en `dickens`; la réplica, con el mismo protocolo, las sitúa entre 0.93 y 1.02 con $R^2 \geq 0.997$ en todas las series. Ningún valor se aproxima a 2, y el ajuste lineal en escala log–log explica casi toda la varianza. Las pendientes menores que 1 de `dickens` son la huella de los costos fijos descritos arriba: el costo por byte cae con n (en la codificación LZW, de 87.85 ns/byte con 16 KiB a 10.34 ns/byte con 8 MiB), de modo que el tiempo crece algo más despacio que n en el rango medido. Las pendientes algo mayores que 1 de la corrida principal en las fuentes sintéticas coinciden con el aumento del costo por byte en los dos mayores tamaños, compatible con efectos de caché y de recolección de memoria, y no se repiten con la misma magnitud en la réplica. La diferencia entre corridas, de hasta 0.13 en una misma serie, es la variabilidad medida del experimento y es mucho menor que la distancia entre un crecimiento lineal y uno cuadrático.

6.2. H2: Huffman frente a la entropía

Predicción. La tasa medida incluye la cabecera y el relleno del último byte, por lo que

$$\mathrm{bpb}_{\mathrm{Huffman}} = \bar{L} + \frac{8 \cdot 262}{n} + \frac{\delta}{n}, \qquad 0 \leq \delta < 8, \tag{6}$$

y, por la ecuación (4), tiende a un valor en $[H_0, H_0 + 1)$ cuando n crece. En E2 con k potencia de dos y $n = 2^{23}$, las frecuencias empíricas son casi iguales, el árbol resultante es completo y equilibrado, y cada símbolo recibe $\log_2 k$ bits: la teoría predice una tasa prácticamente igual a H_0, con un sobrecosto de cabecera de $8 \cdot 262/2^{23}$ bpb, del orden de 2.5×10^{-4}. En la fuente geométrica, en cambio, las probabilidades no son potencias de $1/2$ y cabe esperar una distancia positiva, aunque menor que un bit, respecto de H_0. Sobre texto, Huffman no puede bajar de H_0 porque no modela el contexto.

Evidencia. Figuras 5 y 6 y la columna H_0 de la tabla 11.

Contraste. H2 se acepta. En E2 la tasa es $H_0 + 0.0002$ bpb para todo k, tal como predice la ecuación (6) con un árbol equilibrado. En `skewed`, cuyas probabilidades no son potencias de $1/2$, la distancia con 16 MiB es de 0.027 bpb (4.0923 frente a 4.0657), positiva y muy inferior a un bit. En los 26 archivos del corpus se cumple $\text{bpb}_{\text{Huffman}} < H_0 + 1$. Las distancias mayores tienen dos causas identificables. La primera es la cabecera en archivos diminutos: en `grammar.lsp` (3721 bytes) la cabecera aporta $8 \cdot 262/3721 \approx 0.56$ bpb de los 0.60 bpb de diferencia, y en `xargs.1` (4227 bytes), 0.50 de 0.52 bpb. La segunda es `ptt5`, una imagen de fax en la que un único valor de byte domina el archivo: su entropía es de 1.21 bpb, pero Huffman obtiene 1.66 bpb, porque ningún símbolo puede recibir menos de un bit, por probable que sea. Esta es la limitación clásica de los códigos de longitud entera que motiva la codificación aritmética [23], y no contradice la cota $H_0 + 1$.

6.3. H3: tipo de redundancia y ganador

Predicción. LZW solo ahorra espacio cuando sus frases son largas: cada código cuesta entre 9 y b bits, así que una frase de longitud media m cuesta del orden de b/m bits por byte. En datos con palabras y frases recurrentes, el diccionario captura secuencias largas y la tasa puede quedar por debajo de H_0, algo imposible para un código de orden cero. En datos i.i.d. las frases crecen muy lentamente, porque cada extensión requiere que un par (frase, byte) ya haya aparecido; con bytes uniformes, la mayoría de las frases tienen uno o dos bytes y cada código de 9 a 16 bits representa poco más de un byte, de modo que se espera expansión (bpb > 8). Huffman, por el contrario, nunca supera $\bar{L} < H_0 + 1$ más la cabecera.

Evidencia. Figuras 5 y 6 y tabla 11.

Contraste. H3 se acepta en sus tres partes. En E1, LZW gana con claridad en los datos con frases repetidas —`synthtext` con 16 MiB: 2.21 frente a 4.00 bpb; `dickens` con 8 MiB: 3.29 frente a 4.58—, pierde en la fuente i.i.d. sesgada (`skewed`: 4.74 frente a 4.09) y expande los bytes uniformes (11.00 bpb, frente a 8.0001 de Huffman). E2 confirma el patrón en fuentes sin memoria: LZW es peor que Huffman para todo k, y la brecha crece de 1.13 frente a 1.00 bpb ($k = 2$) a 11.00 frente a 8.00 ($k = 256$). En el corpus, LZW gana en 24 de los 26 archivos, con las mayores ventajas en texto estructurado y repetitivo (`xml`, `nci`, `samba`, `reymont`). Las dos excepciones son coherentes con la hipótesis. `E.coli` es una secuencia de ADN de cuatro letras: su entropía de orden cero (2.00 bpb) ya captura casi toda la redundancia aprovechable, y las repeticiones largas que contiene son demasiado escasas para que un diccionario de 2^{16} frases compense los 16 bits de cada código. En `x-ray`, una radiografía, LZW apenas encuentra frases repetidas —su tasa, 6.67 bpb, supera incluso $H_0 = 6.60$—, y en `sao` —un catálogo binario de estrellas— ambos algoritmos empatan prácticamente. En el agregado del corpus, LZW obtiene 3.30 bpb frente a 5.12 de Huffman.

6.4. H4: sensibilidad del tiempo a la entropía

Predicción. El decodificador canónico ejecuta una iteración por bit leído, $n\bar{L}$ en total; con $\bar{L} \approx \log_2 k$ en E2, su tiempo debería crecer aproximadamente de forma lineal con $\log_2 k$. El codificador de Huffman escribe un campo por símbolo con `BitWriter`, cuyo costo depende solo débilmente de ℓ_s, por lo que se espera menor sensibilidad. La decodificación de LZW escribe cada byte de salida una vez; la entropía solo afecta al número de códigos, que crece cuando las frases se acortan, de modo que se espera una dependencia mucho más débil que la de Huffman.

Evidencia. Figura 7.

Contraste. La parte de H4 referida a Huffman se acepta: con $n = 2^{23}$ fijo, el tiempo de decodificación crece de 8.60 a 57.26 ms y lo hace de forma aproximadamente lineal en $\log_2 k$, que en E2 coincide con $\bar{L}$ (unos 6.95 ms por bit de longitud media). Es exactamente la dependencia $\Theta(n\bar{L})$ del análisis. El rendimiento de decodificación de Huffman en el corpus apunta en la misma dirección: 558 MB/s en `E.coli` ($H_0 = 2.00$) frente a 79 MB/s en `mozilla` ($H_0 = 6.22$). Como se predijo, la codificación de Huffman casi no varía (entre 23.71 y 30.39 ms). La parte referida a LZW se cumple solo en sentido relativo: su decodificación no es insensible a la entropía, pues crece de 13.17 a 42.08 ms, pero lo hace con un factor de 3.2 frente al de 6.7 de Huffman y con mayor dispersión. Esa dependencia no proviene de la longitud de los códigos sino del número de códigos: al subir la entropía, las frases se acortan y hay que resolver más códigos por byte, cada uno con su costo fijo. Lo mismo explica que la codificación de LZW pase de 60.60 a 107.11 ms.

7. Conclusiones

7.1. Comparación entre el análisis teórico y los resultados experimentales

La tabla 12 reúne, para cada aspecto analizado, la predicción derivada en la sección 3 y la evidencia experimental correspondiente.

Tabla 12: Predicción teórica frente a evidencia experimental.

Aspecto	Predicción teórica	Evidencia experimental
Tiempo de Huffman	$\Theta(n)$ al codificar; $\Theta(n\bar{L})$ al decodificar (H1)	Pendiente en `dickens`: 0.94 (cod.), 0.95 (decod.); R^2: 0.994, 0.997; resto en la tabla 10
Tiempo de LZW	$\Theta(n)$ esperado al codificar; $\Theta(n)$ al decodificar (H1)	Pendiente en `dickens`: 0.88 (cod.), 0.85 (decod.); R^2: 0.985, 0.991
Costo por byte	Constante en n	En el mayor prefijo de `dickens`: Huffman 4.53 / 9.59 ns/byte; LZW 10.34 / 3.17 ns/byte (cod. / decod.); decrece con n en tamaños pequeños por costos fijos
Pendientes en todas las series	Cercanas a 1 (H1)	Corrida principal: 0.85–1.08; réplica: 0.93–1.02 ($R^2 \geq 0.997$)
Tasa de Huffman	$[H_0, H_0 + 1)$; cerca de H_0 en fuentes i.i.d. (H2)	E2: 1.00 bpb con $k = 2$ y 8.00 bpb con $k = 256$
Tasa de LZW según el dato	Mejor con frases repetidas; peor en i.i.d.; expansión en bytes uniformes (H3)	E2: 1.13 bpb con $k = 2$ y 11.00 bpb con $k = 256$; E3: gana en 24 de 26 archivos
Corpus completo	Sin predicción universal	Agregado: Huffman 5.12, LZW 3.30 bpb
Decodificación y entropía	Huffman crece con $\bar{L}$; LZW casi insensible (H4)	E2, decodificación: Huffman 8.60 a 57.26 ms; LZW 13.17 a 42.08 ms ($k = 2$ a 256)

Las predicciones de complejidad se confirmaron: los cuatro procedimientos escalan linealmente con n, con pendientes entre 0.85 y 1.08 en la corrida principal y entre 0.93 y 1.02 en la réplica. Las desviaciones respecto de 1 tienen causas identificadas: los costos fijos por llamada —cabecera y construcción del código en Huffman; reserva e inicialización de tablas de tamaño $O(2^b)$ en LZW— reducen la pendiente cuando se amortizan en el rango medido, y los efectos de caché y del recolector la elevan en los mayores tamaños. También se confirmaron las predicciones de tasa: Huffman quedó dentro de $[H_0, H_0 + 1)$ en todos los casos, a solo 0.0002 bpb de H_0 en E2, y LZW ganó con frases repetidas, perdió en fuentes i.i.d. y expandió los bytes uniformes. La predicción que se cumplió solo parcialmente es la insensibilidad de LZW a la entropía: su tiempo de decodificación se multiplicó por 3.2 entre $k = 2$ y $k = 256$, frente al factor de 6.7 de Huffman;

la causa es el mayor número de códigos por byte cuando las frases se acortan. El análisis asintótico tampoco anticipa las constantes, que resultaron decisivas en la práctica: la decodificación bit a bit hace que Huffman decodifique más despacio de lo que codifica, y la búsqueda en la tabla hash por cada byte hace que LZW codifique más despacio de lo que decodifica.

7.2. Respuestas a las preguntas de investigación

RQ1. Sí. Ambos algoritmos escalan en $\Theta(n)$, como predice el análisis, tanto al codificar como al decodificar. La longitud media del código, $\bar{L}$, no altera el orden de crecimiento de Huffman, pero sí su constante: con n fijo, su tiempo de decodificación crece linealmente con $\log_2 k$. Las constantes difieren entre operaciones: en el mayor tamaño, Huffman codifica a 4.50–6.26 ns/byte y decodifica a 9.14–11.18, mientras que LZW codifica a 10.34–14.72 y decodifica a 3.17–5.11. En la práctica, Huffman es preferible cuando la velocidad de codificación es prioritaria, y LZW cuando lo es la de decodificación.

RQ2. Huffman depende solo de la distribución de frecuencias y queda siempre en $[H_0, H_0 + 1)$; su límite es un bit por símbolo, visible en `ptt5`. LZW depende de la repetición de frases: puede bajar mucho de H_0 en texto y datos estructurados (`xml`: 1.66 bpb frente a $H_0 = 5.52$), pero pierde frente a Huffman en fuentes sin memoria y expande los datos aleatorios. Se recomienda Huffman para datos cercanos a una fuente sin memoria con distribución sesgada y LZW para texto, código fuente, registros y formatos estructurados.

La tabla 13 resume estas respuestas como criterio práctico de elección.

Tabla 13: Algoritmo recomendado según la situación, con la evidencia medida.

Situación	Recomendado	Evidencia
Texto, código fuente y formatos estructurados	LZW	`xml`: 1.66 bpb frente a 5.55 de Huffman
Datos sin frases repetidas y con frecuencias sesgadas	Huffman	`skewed`: 4.09 bpb frente a 4.74 de LZW
Datos aleatorios o ya comprimidos	Huffman	Bytes uniformes: 8.00 bpb; LZW los expande a 11.00
Prioridad en la velocidad de compresión	Huffman	4.50–6.26 ns/byte frente a 10.34–14.72 de LZW
Prioridad en la velocidad de descompresión	LZW	3.17–5.11 ns/byte frente a 9.14–11.18 de Huffman

En términos generales, el análisis teórico indica que la elección entre Huffman y LZW no depende de su complejidad asintótica, que es lineal en ambos casos, sino de la coincidencia entre la redundancia que cada uno explota y la que contienen los datos, en línea con la observación de las revisiones del área [10, 11]. Los resultados confirman esta conclusión y la matizan en dos aspectos. Primero, a igual orden de complejidad, las constantes de cada operación difieren mucho —de 3.17 a 14.72 ns/byte en el mayor tamaño— y siguen asimetrías opuestas, de modo que la elección también depende de si importa más comprimir o descomprimir rápido. Segundo, el tipo de redundancia decide el ganador con mucha claridad —LZW en 24 de 26 archivos, Huffman en todas las fuentes sin memoria—, de modo que conviene conocer el dato antes de elegir el algoritmo.

Referencias

[1] K. Sayood, *Introduction to Data Compression*, 5th ed. Morgan Kaufmann, 2017.

[2] D. Salomon and G. Motta, *Handbook of Data Compression*, 5th ed. London: Springer, 2010.

[3] C. E. Shannon, "A mathematical theory of communication," *Bell System Technical Journal*, vol. 27, no. 3, pp. 379–423, Jul. 1948.

[4] D. A. Huffman, "A method for the construction of minimum-redundancy codes," *Proceedings of the IRE*, vol. 40, no. 9, pp. 1098–1101, Sep. 1952.

[5] T. A. Welch, "A technique for high-performance data compression," *Computer*, vol. 17, no. 6, pp. 8–19, Jun. 1984.

[6] J. Ziv and A. Lempel, "Compression of individual sequences via variable-rate coding," *IEEE Transactions on Information Theory*, vol. 24, no. 5, pp. 530–536, Sep. 1978.

[7] P. Deutsch, "DEFLATE compressed data format specification version 1.3," RFC 1951, RFC Editor, May 1996. [Online]. Available: https://www.rfc-editor.org/info/rfc1951

[8] A. B. Jambek and N. A. Khairi, "Performance comparison of Huffman and Lempel-Ziv Welch data compression for wireless sensor node application," *American Journal of Applied Sciences*, vol. 11, no. 1, pp. 119–126, 2014.

[9] F. Bjärås, "Comparative study of compression algorithms on time series data for IoT devices," Master's thesis, Department of Electrical and Information Technology, Lund University, 2019. [Online]. Available: https://lup.lub.lu.se/luur/download?fileOId=8989660&func=downloadFile&recordOId=8989654

[10] T. Bell, I. H. Witten, and J. G. Cleary, "Modeling for text compression," *ACM Computing Surveys*, vol. 21, no. 4, pp. 557–591, Dec. 1989.

[11] U. Jayasankar, V. Thirumal, and D. Ponnurangam, "A survey on data compression techniques: From the perspective of data quality, coding schemes, data type and applications," *Journal of King Saud University – Computer and Information Sciences*, vol. 33, no. 2, pp. 119–140, 2021.

[12] M. N. Fauzan, M. Alif, and C. Prianto, "Comparison of Huffman algorithm and Lempel Ziv Welch algorithm in text file compression," *IT Journal Research and Development*, vol. 7, no. 2, pp. 155–169, 2022.

[13] M. H. Btoush, J. Siddiqi, B. Akhgar, and Z. Dawahdeh, "Observations on compressing text files of varying length," in *Fifth International Conference on Information Technology: New Generations (ITNG 2008)*, 2008, pp. 224–228.

[14] A. Georges, D. Buytaert, and L. Eeckhout, "Statistically rigorous Java performance evaluation," in *Proceedings of the 22nd ACM SIGPLAN Conference on Object-Oriented Programming Systems, Languages and Applications (OOPSLA '07)*, 2007, pp. 57–76.

[15] C. C. McGeoch, *A Guide to Experimental Algorithmics*. Cambridge University Press, 2012.

[16] R. Arnold and T. Bell, "A corpus for the evaluation of lossless compression algorithms," in *Proceedings DCC '97. Data Compression Conference*. Snowbird, UT, USA: IEEE Computer Society Press, 1997, pp. 201–210.

[17] S. Deorowicz, "Universal lossless data compression algorithms," Ph.D. dissertation, Silesian University of Technology, Faculty of Automatic Control, Electronics and Computer Science, Gliwice, Poland, 2003. [Online]. Available: https://sun.aei.polsl.pl/~sdeor/pub/deo03.pdf

[18] R. G. Gallager, "Variations on a theme by Huffman," *IEEE Transactions on Information Theory*, vol. 24, no. 6, pp. 668–674, Nov. 1978.

[19] D. E. Knuth, "Dynamic Huffman coding," *Journal of Algorithms*, vol. 6, no. 2, pp. 163–180, Jun. 1985.

[20] J. S. Vitter, "Design and analysis of dynamic Huffman codes," *Journal of the ACM*, vol. 34, no. 4, pp. 825–845, Oct. 1987.

[21] A. Moffat and A. Turpin, "On the implementation of minimum redundancy prefix codes," *IEEE Transactions on Communications*, vol. 45, no. 10, pp. 1200–1207, 1997.

[22] A. Moffat, "Huffman coding," *ACM Computing Surveys*, vol. 52, no. 4, pp. 85:1–85:35, Aug. 2019.

[23] I. H. Witten, R. M. Neal, and J. G. Cleary, "Arithmetic coding for data compression," *Communications of the ACM*, vol. 30, no. 6, pp. 520–540, Jun. 1987.

[24] J. Ziv and A. Lempel, "A universal algorithm for sequential data compression," *IEEE Transactions on Information Theory*, vol. 23, no. 3, pp. 337–343, May 1977.

[25] J. A. Storer and T. G. Szymanski, "Data compression via textual substitution," *Journal of the ACM*, vol. 29, no. 4, pp. 928–951, Oct. 1982.

[26] X. Cheng, B. Li, J. Wu, W. Li, C. Luo, and J. Su, "A novel dictionary management scheme of LZW compression algorithm based on insignificant dictionary area," in *2021 IEEE 6th International Conference on Signal and Image Processing (ICSIP)*, Nanjing, China, 2021, pp. 975–979.

[27] D. S. Johnson, "A theoretician's guide to the experimental analysis of algorithms," in *Data Structures, Near Neighbor Searches, and Methodology: Fifth and Sixth DIMACS Implementation Challenges*, ser. DIMACS Series in Discrete Mathematics and Theoretical Computer Science. Providence, RI: American Mathematical Society, 2002, vol. 59, pp. 215–250.

[28] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 4th ed. Cambridge, MA: MIT Press, 2022.

A. Reproducibilidad

El repositorio contiene todo lo necesario para repetir el estudio. Desde la raíz, con `bash`, la secuencia completa es la del listado 3. El archivo `env.sh` configura el JDK, Apache Ant y Tectonic incluidos en `.tools/`; el *script* de corpus descarga Canterbury, el corpus grande y Silesia, y verifica sus sumas SHA-256; `ant benchmark` ejecuta E1–E3 y escribe en `results/` los CSV, la tabla del corpus y las macros que este informe importa; `build.sh` copia esos resultados y compila el PDF.

```
source env.sh # JDK, Ant y Tectonic portables
data/download-corpora.sh # corpus Canterbury, grande y Silesia (SHA-256)
cd code
ant test # 43 pruebas JUnit
ant benchmark # experimentos E1-E3 -> ../results
cd ../report
./build.sh # copia ../results y genera build/main.pdf
```

Listado 3: Comandos para reproducir el estudio.

Para una verificación rápida del arnés, sin valor estadístico, puede ejecutarse `ant benchmark -Dbench.args="--quick"`, y la opción `--only e1,e2` restringe los experimentos. El proyecto también se abre directamente en NetBeans (*File* > *Open Project*, carpeta `code/`), donde *Run* ejecuta una demostración de ambos algoritmos y *Test* ejecuta la batería de pruebas. La interfaz de línea de comandos permite usar los algoritmos sobre archivos arbitrarios:

```
java -jar dist/HuffmanVsLZW.jar compress <huffman|lzw|lzw-N> <entrada> <salida>
java -jar dist/HuffmanVsLZW.jar decompress <entrada> <salida>
java -jar dist/HuffmanVsLZW.jar stats <entrada>
```

B. Detalle de la implementación

A continuación se describe cada componente del trabajo: su objetivo, su funcionamiento, cómo se implementó y cómo se integra con el resto.

B.1. Codificación de Huffman

Objetivo. Implementar un compresor de Huffman estático y canónico sobre el alfabeto de bytes que sea correcto para cualquier entrada, que alcance la longitud media óptima (ecuación (4)) y cuya cabecera sea compacta y verificable.

Cómo funciona. La codificación hace dos pasadas: la primera cuenta las frecuencias de los 256 bytes; con ellas se calculan las longitudes óptimas mediante el algoritmo voraz (algoritmo 1) y se asignan códigos canónicos (ecuación (5)); la segunda pasada escribe el código de cada byte. La decodificación lee las 256 longitudes de la cabecera, reconstruye las tablas canónicas y decodifica bit a bit (algoritmo 2).

Cómo se implementó. `HuffmanCodec` usa un `PriorityQueue` de índices de nodo con desempate por índice, arreglos planos de pesos y padres (sin objetos nodo), y un recorrido descendente para las profundidades (listado 1). El formato del flujo es: magia `HF`, longitud original de 4 bytes y 256 longitudes de 1 byte, seguidas de los bits. La tabla canónica rechaza longitudes sobresuscritas (suma de Kraft mayor que 1), lo que detecta cabeceras corruptas. `BitWriter` y `BitReader` manejan campos de 1 a 64 bits con un acumulador de 64 bits, bit más significativo primero.

Integración. `HuffmanCodec` implementa la interfaz `Codec`, de modo que el arnés (`BenchmarkRunner`) y la línea de comandos (`Main`) lo usan sin conocer sus detalles. `BitWriter` y `BitReader` son compartidos con LZW. Sus pruebas (`HuffmanCodecTest`, 11; `BitStreamTest`, 5) forman parte de `ant test`.

Archivos. `code/src/compression/huffman/HuffmanCodec.java`, `code/src/compression/io/BitWriter.java`, `code/src/compression/io/BitReader.java` y sus pruebas en `code/test/compression/huffman/` y `.../io/`; en el informe, sección 3.2.

B.2. Algoritmo LZW

Objetivo. Implementar LZW con códigos de ancho variable y tamaño máximo de diccionario configurable, de forma que la memoria quede acotada.

Cómo funciona. El codificador extiende la frase actual mientras el par (frase, byte) esté en el diccionario y, cuando no lo está, emite el código de la frase y registra la nueva entrada (algoritmo 3). Los códigos crecen de 9 a b bits; al llenarse el diccionario se emite `CLEAR` (código 256) y se reinicia. El decodificador reconstruye el mismo diccionario con un código de retraso, resuelve el caso $KwKwK$ y escribe cada frase de atrás hacia adelante siguiendo los prefijos (algoritmo 4).

Cómo se implementó. `LzwCodec` representa el diccionario del codificador con `PhraseTable`, una tabla hash de direccionamiento abierto de capacidad 2^{b+1} (factor de carga máximo 0.5), sondeo lineal y hashing de Fibonacci; la búsqueda devuelve la ranura donde insertar si la clave no existe (listado 2). El decodificador usa cuatro arreglos indexados por código (`prefix`, `suffix`, `firstByte`, `phraseLength`). El formato es: magia `LZ`, b (1 byte) y longitud original (4 bytes), seguidos de los códigos. El decodificador valida cada código frente al tamaño actual del diccionario y la longitud declarada, y rechaza flujos corruptos. El método `traceCodes` expone la secuencia emitida para pruebas y enseñanza.

Integración. `LzwCodec` implementa `Codec` y reutiliza `BitWriter` y `BitReader`. El arnés lo instancia con $b = 16$ en E1–E3; la línea de comandos acepta `lzw` o `lzw-N`. Sus 8 pruebas (`LzwCodecTest`) forman parte de `ant test`.

Archivos. `code/src/compression/lzw/LzwCodec.java` y `code/test/compression/lzw/LzwCodecTest.java`; en el informe, sección 3.3.

B.3. Metodología experimental y arnés de medición

Objetivo. Medir ambos algoritmos de forma estadísticamente rigurosa y reproducible, y producir automáticamente los datos (CSV, tabla del corpus y macros) que alimentan el informe.

Cómo funciona. `BenchmarkRunner` ejecuta E1–E3 según el protocolo de la sección 4.6: calentamiento global, verificación de ida y vuelta, calentamiento local, repeticiones cronometradas y resúmenes estadísticos. `Stats` calcula media, mediana, desviación estándar, intervalo de confianza del 95 % con la t de Student, el ajuste log–log por mínimos cuadrados y la entropía de orden cero. `DataGenerator` produce las fuentes sintéticas deterministas de la tabla 7.

Cómo se implementó. El arnés trabaja sobre la interfaz `Codec`, lo que permite medir igual a Huffman y a LZW. `ResultWriter` escribe los CSV, la tabla del corpus en LaTeX y `macros.tex`, de modo que ningún número se transcribe a mano. `Main` ofrece la línea de comandos con códigos de salida 0, 1 y 2. El *script* `data/download-corpora.sh` descarga y verifica los corpus.

Integración. El arnés consume los códecs de Huffman y LZW; sus salidas en `results/` son la entrada de las figuras y tablas del informe (anexo B.4). Sus pruebas (`StatsTest`, 6; `DataGeneratorTest`, 3; `BenchmarkRunnerTest`, 4; `MainTest`, 6) forman parte de `ant test`.

Archivos. El paquete `code/src/compression/bench/` (`BenchmarkRunner`, `Stats`, `DataGenerator`, `ResultWriter`), la clase `Main` en `code/src/compression/app/`, sus pruebas y el *script* `data/download-corpora.sh`; en el informe, secciones 3.4 y 3.5 y sección 4.

B.4. Generación de figuras y tablas

Objetivo. Convertir las mediciones del arnés en las figuras y tablas de E1–E3, de modo que el informe se genere a partir de los datos sin transcribir números a mano.

Cómo funciona. El informe consume las salidas de `BenchmarkRunner`: `results/e1_*.csv` (tiempos, costo por byte y tasas de E1 por fuente), `fits.csv` y `replica/fits.csv` (pendientes log–log y R^2 de la corrida principal y de la réplica), `e2_entropy.csv`, `e3_corpus.csv`, `macros.tex` y `table_corpus.tex`. A partir de ellas se construyen las figuras y tablas de la sección 5 y la tabla 12.

Cómo se implementó. Las figuras se dibujan con `pgfplots` directamente desde los CSV y las tablas se leen con `pgfplotstable` o se importan de `table_corpus.tex`; los valores citados en el texto usan las macros `\res*` de `macros.tex` cuando existen. Cada número escrito en la prosa se validó contra el CSV correspondiente. El paso de copia de `report/build.sh` trae una instantánea de `results/` al informe, incluidas las pendientes de la réplica como `fits_replica.csv`.

Integración. Las figuras y tablas dependen de las salidas del arnés (anexo B.3); la discusión de las hipótesis H1–H4 (sección 6) y las conclusiones (sección 7) se apoyan en ellas.

Archivos. `report/sections/05-resultados.tex`, `results/` y el paso de copia de `report/build.sh`; en el informe, sección 5.